

UNIVERSITÀ POLITECNICA DELLE MARCHE

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA
INFORMATICA E DELL'AUTOMAZIONE

Progetto Zero Trust Architecture 2026

*Protezione Adattiva di Database e API con Verifica Continua,
Machine Learning e Micro-Segmentazione*

Candidati:

Andrea Flaiani (Matr. 1126928)
Andrea Altieri (Matr. 1128865)
Niccolò de Pascali (Matr. 1123958)
Matteo Risolo (Matr. 1122743)
Simone Murazzo (Matr. 1113295)

Docente:

Prof. Luca Spalazzi

Indice

1	Introduzione e obiettivi	5
1.1	La protezione dei dati aziendali e delle risorse critiche	5
1.2	La filosofia Zero Trust: “never Trust, always Verify”	5
1.3	Il caso di studio: l’infrastruttura ospedaliera	6
1.4	Obiettivi del progetto	6
1.5	Struttura della relazione	7
2	Architettura di sistema e microsegmentazione	8
2.1	Modellazione delle minacce e analisi del rischio	8
2.2	Le quattro reti segregate	8
2.3	Il choke-point obbligato: envoy proxy	11
2.4	La tupla multidimensionale Zero Trust (ZTA 6D)	12
2.5	Orchestrazione e Isolamento dei Container	12
2.6	Il server database di risorsa (MongoDB)	13
2.7	Le Web API e il portale client di simulazione	13
3	Implementazione pratica e configurazione	15
3.1	La microsegmentazione e il deploy con docker-compose	15
3.2	La configurazione di Envoy Proxy come PEP	15
3.3	Formalizzazione del Controllo degli Accessi: Modello ABAC e Matrice Dinamica	18
3.4	Il motore decisionale OPA (PDP) e l’integrazione Splunk MLTK	20
3.5	Regole di network protection e tracciamento su nftables	22
3.6	Autenticazione crittografica: mTLS, TPM e fingerprinting JA3	24
3.6.1	Mutua autenticazione TLS (mTLS)	24
3.6.2	Attestazione hardware TPM	24
3.6.3	Fingerprinting software JA3	26
3.7	Rilevamento delle intrusioni di rete: Snort NIDS	26
3.8	Aggregazione centralizzata dei log: Fluent-Bit	28
3.9	Il modello predittivo di Machine Learning: Splunk MLTK e Gradient Boosting	29
3.9.1	Dall’approccio statico alla valutazione comportamentale continua	29
3.9.2	Generazione del dataset e modello matematico	29
3.9.3	Selezione dell’algoritmo: Gradient Boosting vs Random Forest	29
3.9.4	Addestramento e Regressione in tempo reale	30
4	Simulazione degli scenari e validazione	31
4.1	Scenario 1: accesso legittimo	31
4.2	Scenario 2: dispositivo non autorizzato	33
4.3	Scenario 3: accesso remoto negato	36
4.4	Scenario 4: tentativo di bypass L4 e rilevamento attivo	38
4.5	Scenario 5: protezione dai payload dannosi a livello applicativo (L7)	40
4.5.1	Simulazione di compromissione del dispositivo (Bypass L7)	41

5 Conclusioni e sviluppi futuri	44
5.1 Obiettivi raggiunti	44
5.2 Sviluppi futuri	44

Elenco delle figure

1	Topologia di rete dell'infrastruttura Zero Trust con flussi logici e di telemetria.	9
2	Diagramma dei flussi informativi e dei componenti della Zero Trust Architecture.	11
3	Schermata del portale di login protetto dell'ospedale San Raffaele. . . .	31
4	Interfaccia del database pazienti visualizzata con successo da Alice dopo validazione TPM e rischio minimo.	32
5	Log di autorizzazione (ALLOW) per l'accesso di Alice indicizzato in Splunk, comprensivo del risk score comportamentale calcolato (≈ 9.96).	32
6	Mappatura logica dei flussi nello Scenario 1 (Accesso legittimo di Alice).	33
7	Schermata di errore 403 Forbidden mostrata a Bob per mancata attestazione hardware del chip TPM.	34
8	Log di diniego (DENY) per l'accesso di Bob indicizzato in Splunk, evidenziante l'assenza della firma hardware TPM (<code>tpm_present: false</code>).	35
9	Mappatura logica dei flussi nello Scenario 2 (Blocco al login di Bob per mancanza di TPM).	35
10	Schermata di errore 403 Forbidden mostrata a Charlie a causa del tentativo di login remoto non consentito.	36
11	Log di diniego (DENY) per l'accesso remoto di Charlie indicizzato in Splunk, evidenziante la mancata conformità della localizzazione di rete (<code>network_internal: false</code>).	37
12	Mappatura logica dei flussi nello Scenario 3 (Blocco al login di Charlie da remoto).	37
13	Schermata del pannello di ricerca di Splunk che mostra il log di blocco [NFT-BLOCK] indicizzato.	39
14	Mappatura logica dei flussi nello Scenario 4 (Tentativo di bypass e isolamento).	39
15	Schermata di errore 403 Forbidden visualizzata a seguito del blocco di una query MongoDB dannosa a livello L7.	41
16	Log di blocco (response code 403) relativo alla richiesta L7 di Alice in Splunk, che evidenzia l'intercettazione del metodo DELETE sulla risorsa <code>/api/patients</code>	41
17	Log di decisione DENY indicizzato in Splunk per il tentativo di accesso diretto al database da parte di Alice (PC compromesso), con risk score pari a ≈ 95.49	43
18	Mappatura logica dei flussi nello Scenario 5 (Blocco di una query MongoDB dannosa a livello L7).	43

Elenco dei listati

1	Definizione delle quattro reti segregate nel file docker-compose.yaml. . .	10
2	Definizione delle quattro reti segregate nel file docker-compose.yaml. . .	15
3	Estratto della configurazione del listener MongoDB in envoy.yaml. . . .	17
4	Configurazione del filtro HTTP Lua in envoy.yaml per la prima linea di difesa L7.	18
5	Struttura della query Splunk MLTK generata dinamicamente da OPA. . .	20
6	Regole di ispezione profonda L7 DPI per confidenzialità e integrità in rules.rego.	21
7	Regole di autorizzazione adattiva (Score-based) e short-circuiting (Criteria-based) in rules.rego.	22
8	Configurazione delle catene di filtering e NAT nello script di avvio del firewall.	23
9	Generazione dei certificati client con estensione TPM simulata.	25
10	Validazione dell'estensione TPM nel certificato client da parte di OPA. . .	25
11	Regole di rilevamento avanzate (Volumetriche, Protocolлари e Crittografiche) per Snort.	27
12	Query SPL per l'addestramento del modello Gradient Boosting su Splunk MLTK.	30
13	Simulazione di attacco bypass diretto tramite curl dal terminale di Charlie.	38
14	Dettaglio del log di blocco generato da nftables per il tentativo di bypass.	38
15	Simulazione di accesso diretto al database MongoDB dal terminale del dispositivo compromesso di Alice, utilizzando i certificati mTLS originali per autenticarsi verso Envoy sulla porta 27017.	42

CAPITOLO 1

Introduzione e obiettivi

In questo primo capitolo viene presentato il contesto applicativo e metodologico alla base del progetto, definendo le sfide di sicurezza tipiche della gestione di database distribuiti e illustrando i principi teorici del paradigma Zero Trust. Si introduce quindi il caso di studio ospedaliero scelto come dominio di riferimento e si delineano gli obiettivi e la struttura complessiva della relazione.

1.1 La protezione dei dati aziendali e delle risorse critiche

Le infrastrutture IT aziendali rappresentano uno dei bersagli più ambiti e critici per gli attori malevoli a livello globale. La digitalizzazione dei servizi e la transizione verso sistemi in cloud hanno introdotto enormi benefici in termini di efficienza operativa, tra cui l'accesso condiviso a risorse sensibili, l'automazione dei flussi e il monitoraggio IoT di asset industriali. Tuttavia, questa evoluzione ha contemporaneamente esteso a dismisura la superficie d'attacco delle organizzazioni.

La criticità di questo scenario è dettata da molteplici fattori. In primo luogo, vi è la natura estremamente riservata delle informazioni trattate all'interno dei database, che includono dati finanziari, proprietà intellettuali, credenziali e profili di utenti. La perdita di riservatezza o l'alterazione di tali dati viola gravemente le principali normative vigenti sulla protezione dei dati personali e sulla sicurezza delle reti a livello internazionale. In secondo luogo, va considerato l'impatto sulla **business continuity**: a differenza dei semplici disservizi temporanei, un attacco informatico di tipo ransomware che cripta o altera i database operativi può causare il blocco completo dei servizi, enormi perdite finanziarie e danni reputazionali incalcolabili. Infine, l'eterogeneità degli accessi complica ulteriormente il quadro: gli operatori e gli amministratori devono poter consultare i dati sia dai terminali interni alla rete aziendale, come le workstation d'ufficio, sia da remoto, ad esempio in regime di smart working, introducendo vettori di minaccia esterni non controllati.

1.2 La filosofia Zero Trust: “never Trust, always Verify”

La sicurezza perimetrale classica, basata sul modello a “castello”, in cui un firewall protegge una rete interna considerata intrinsecamente sicura da una rete esterna ostile, si è dimostrata obsoleta e inefficace. Una volta che un attaccante riesce a penetrare il perimetro, tramite phishing, exploit o **insider threat**, ottiene totale libertà di movimento laterale.

Il paradigma **Zero Trust Architecture** (ZTA), codificato dal NIST nel documento speciale *SP 800-207*, supera questa concezione abbattendo il concetto di fiducia implicita basata sulla localizzazione di rete. I cardini della filosofia ZTA sono tre:

1. **Mai fidarsi, verificare sempre (Never Trust, Always Verify)**: ogni singola richiesta di accesso ad una risorsa, indipendentemente dalla rete da cui proviene, sia essa interna o esterna, deve essere autenticata, autorizzata e validata crittograficamente prima dell'inoltro.

2. **Accesso con privilegi minimi (Least Privilege):** l'accesso viene concesso in modo estremamente granulare e ristretto solo alle risorse necessarie per lo svolgimento della specifica attività, ad esempio limitando la scrittura e la cancellazione di record sensibili ai soli operatori autorizzati.
3. **Presumere la violazione (Assume Breach):** progettare il sistema assumendo che alcuni segmenti siano già compromessi. Ciò impone l'adozione di micro-segmentazione di rete per limitare il raggio d'azione dell'attaccante, crittografia end-to-end per proteggere i dati in transito e sistemi di monitoraggio continuo del traffico.

1.3 Il caso di studio: l'infrastruttura ospedaliera

Il dominio applicativo scelto per la validazione dell'architettura è quello di una **struttura ospedaliera**, un contesto in cui la criticità dei dati trattati e l'eterogeneità degli accessi rendono particolarmente evidente la necessità di un approccio Zero Trust.

Il sistema gestisce un database MongoDB contenente un esempio di informazioni sanitarie altamente sensibili, come anagrafiche dei pazienti, cartelle cliniche e dati diagnostici. L'accesso a queste risorse è richiesto da tre categorie di operatori con profili di rischio differenti:

- **Alice:** personale medico interno all'ospedale, dotato di workstation aziendale con chip **TPM** hardware e collegato dalla rete locale (10.0.1.0/24). Rappresenta l'utente a rischio minimo.
- **Bob:** operatore interno che utilizza un dispositivo personale privo di attestazione hardware TPM. Pur collegandosi dalla rete aziendale, la mancanza di garanzie sull'integrità del dispositivo lo sottopone a soglie di rischio più restrittive. Visto da un altro punto di vista, Bob rappresenta un esempio di **insider threat** potenziale, per esempio un utente che ha rubato le credenziali ma non possiede TPM.
- **Charlie:** professionista in regime di smart working, dotato di certificato e chip TPM ma collegato da una rete esterna non controllata (192.168.100.0/24). La provenienza da una rete non fidata impone politiche di tolleranza al rischio più restrittive.

Questo scenario consente di dimostrare concretamente come l'architettura adatti dinamicamente le proprie soglie di sicurezza in funzione del contesto multidimensionale di ciascun accesso, combinando l'identità dell'utente, la postura del dispositivo e la localizzazione di rete.

1.4 Obiettivi del progetto

Il presente progetto si pone l'obiettivo di realizzare un'architettura Zero Trust completa, funzionante ed integrata per la protezione di un database MongoDB aziendale. Il sistema è progettato per operare una valutazione continua e adattiva del rischio (*Continuous Authorization*) basata sul contesto di rete, sul comportamento dell'utente e sulla postura di sicurezza del dispositivo, inclusa rigorosamente l'attestazione hardware tramite modulo TPM.

Per raggiungere questo scopo, vengono impiegate diverse tecnologie integrate. In primo luogo, envoy opera come **Policy Enforcement Point** (PEP), intercettando ogni singola richiesta HTTP o MongoDB in modo sicuro mediante crittografia mutua TLS (*mTLS*), eseguendo il fingerprinting software (*JA3*) e l'ispezione profonda dei pacchetti a livello applicativo (*L7 DPI*). In secondo luogo, open policy agent (OPA) funge da **Policy Decision Point** (PDP) decentralizzato, valutando le richieste in tempo reale sulla base delle sei dimensioni Zero Trust (ZTA 6D), rappresentate dai parametri *u* (user), *d* (device), *s* (software), *n* (network), *a* (action), *r* (resource).

L'analisi comportamentale è affidata a splunk Machine Learning Toolkit (MLTK), interpellato sincronicamente da OPA per calcolare un punteggio di rischio dinamico tramite modelli predittivi basati sull'algoritmo **Gradient Boosting**. Parallelamente, splunk SIEM si occupa dell'ingestion asincrona dei log per il monitoraggio e la correlazione storica degli eventi. Infine, a livello di rete, nftables e snort (NIDS) garantiscono rispettivamente la microsegmentazione del traffico a livello L3/L4 e il rilevamento tempestivo di tentativi di movimento laterale o bypass del proxy.

1.5 Struttura della relazione

Il documento è organizzato come segue: il Capitolo 2 illustra l'architettura complessiva della topologia di rete a quattro zone. Il Capitolo 3 scende nel dettaglio dell'implementazione tecnica di ciascun componente di sicurezza, descrivendo il funzionamento di envoy, OPA, splunk, snort e nftables. Il Capitolo 4 descrive la validazione del sistema attraverso la simulazione di scenari d'attacco ed accessi legittimi. Infine, il Capitolo 5 riassume i risultati e propone futuri sviluppi progettuali.

CAPITOLO 2

Architettura di sistema e microsegmentazione

In questo capitolo viene analizzata l'architettura logica e fisica dell'infrastruttura, descrivendo la segmentazione di rete a quattro zone e il ruolo di envoy come unico punto di transito autorizzato per il traffico dati.

2.1 Modellazione delle minacce e analisi del rischio

In conformità con la metodologia di analisi del rischio discussa nel corso di *Advanced Cybersecurity*, l'architettura adotta un framework di *Threat Modeling* che classifica i componenti secondo la loro criticità. Gli **Asset** (O) da proteggere includono il database MongoDB, le cartelle cliniche sensibili e gli endpoint delle Web API. Le **Vulnerabilità** (V) identificate comprendono l'esposizione di terminali sprovvisti di chip TPM, l'utilizzo di protocolli non cifrati e le anomalie comportamentali degli utenti.

Le **Minacce** sono modellate come eventi in grado di sfruttare V per compromettere l'integrità o la confidenzialità di O . Il sistema adotta una difesa in profondità, distinguendo tra meccanismi di *prevenzione* (volti a inibire il verificarsi di una minaccia, es. OPA e mTLS) e meccanismi di *rilevamento* (volti a notificare violazioni che hanno superato il primo strato preventivo, es. Snort e Splunk). Il *Trust Score* prodotto dal modello MLTK viene interpretato come una quantificazione dinamica della *Likelihood* (L) di occorrenza di un attacco: il rischio operativo R è espresso dalla funzione $R = L \times I$, dove I rappresenta l'impatto critico della risorsa. Il sistema accetta la transazione solo se R è inferiore al *rischio accettabile per il business* (cfr. 5.13 materiale didattico).

2.2 Le quattro reti segregate

La sicurezza dell'infrastruttura si basa sul principio della **microsegmentazione**, implementata definendo quattro reti virtuali isolate tramite Docker. Questa compartimentazione riduce drasticamente l'area di impatto di un'eventuale compromissione. La segmentazione fisica ed i relativi indirizzamenti IP delle quattro aree protette sono schematizzati visivamente nella Figura 1, mentre la loro definizione a livello di infrastruttura è dettagliata nella configurazione mostrata nel Listato 1.

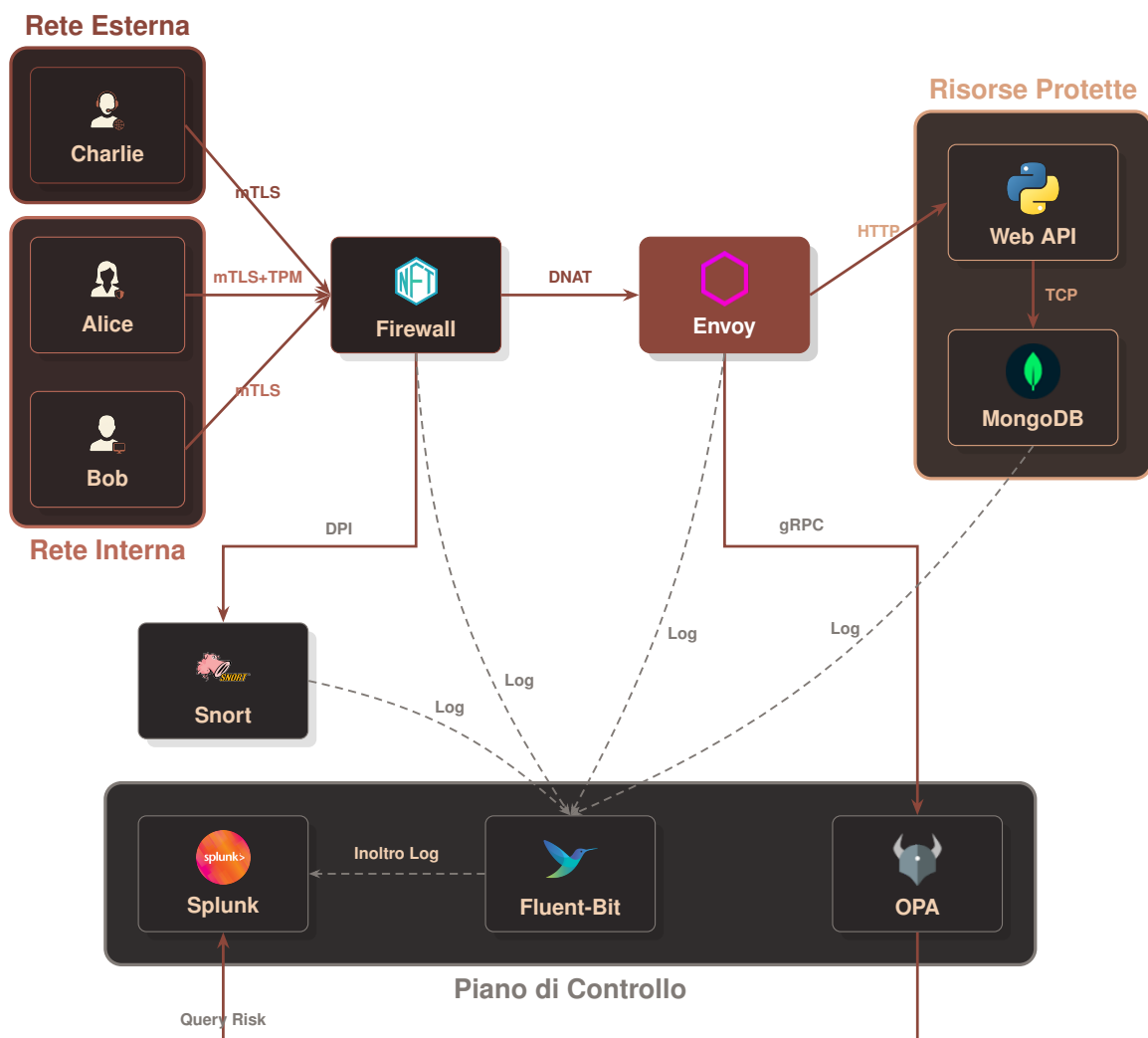


Figura 1: Topologia di rete dell'infrastruttura Zero Trust con flussi logici e di telemetria.

 docker-compose.yaml

```
1 networks:
2   frontend-net:
3     driver: bridge
4     ipam:
5       config:
6         - subnet: 10.0.1.0/24
7   backend-net:
8     driver: bridge
9   control-plane-net:
10    driver: bridge
11   external-net:
12     driver: bridge
13     ipam:
14       config:
15         - subnet: 192.168.100.0/24
```

Listing 1: Definizione delle quattro reti segregate nel file docker-compose.yaml.

La prima area è la **zona untrusted** (external-net), che rappresenta la rete esterna non controllata da cui provengono le richieste di utenti remoti. In questa zona opera, ad esempio, il client dell'operatore in smart working. Non vi è alcuna rotta di instradamento che colleghi direttamente questa zona al database o ai servizi interni: l'unico punto di contatto esposto è il listener di envoy, rigorosamente protetto da crittografia.

La seconda area è la **zona DMZ o frontend** (frontend-net), assimilabile alla rete locale fisica dell'azienda. Qui risiedono i client interni utilizzati dal personale d'ufficio e il firewall nftables. Anche in questo caso, i client del frontend non possiedono le rotte di rete dirette per raggiungere il backend: il firewall intercetta il traffico a livello L3/L4 e ne forza l'instradamento verso il proxy (envoy), unico attore preposto all'ispezione e all'autorizzazione delle richieste.

La terza area è il **secure core o backend** (backend-net), una rete altamente protetta e completamente isolata. All'interno di questa zona sono collocati il database mongodb e le Web API del servizio aziendale. Nessun client, sia esso esterno o interno, può avviare una connessione diretta con queste macchine: l'unico container autorizzato a inoltrare pacchetti nel backend è il proxy, che agisce da intermediario per le sole richieste esplicitamente validate dal motore decisionale. In merito alla scelta del database, le specifiche del progetto consentivano l'adozione di mongodb o, in alternativa, di una configurazione composta da proxysql e mysql. In questa architettura si è preferito utilizzare il database orientato ai documenti mongodb per agevolare l'ispezione granulare dei comandi BSON nativi direttamente a livello applicativo (*L7 DPI*), sfruttando il filtro di decodifica specifico di envoy (envoy.filters.network.mongo.proxy).

La quarta e ultima area è la **control plane o trust zone** (control-plane-net). Si tratta di una rete di gestione isolata in cui risiedono i sistemi decisionali e di monitoraggio, tra cui il motore delle regole OPA, il raccogliatore di log fluent-bit e il SIEM splunk. Questa zona comunica esclusivamente con il proxy per fornire in tempo reale i verdeti di validazione delle richieste e con tutti i container per la raccolta asincrona della telemetria in background.

2.3 Il choke-point obbligato: envoy proxy

Al centro dell'intera topologia si colloca **envoy**, che agisce come **Policy Enforcement Point** (PEP) e rappresenta il punto di strozzatura obbligato per tutto il traffico. Qualsiasi tentativo di comunicazione che cerchi di scavalcare questo passaggio viene intercettato e scartato a livello di rete dal firewall. Il posizionamento strategico del PEP e il flusso dei messaggi di autorizzazione verso gli altri moduli sono illustrati nella Figura 2.

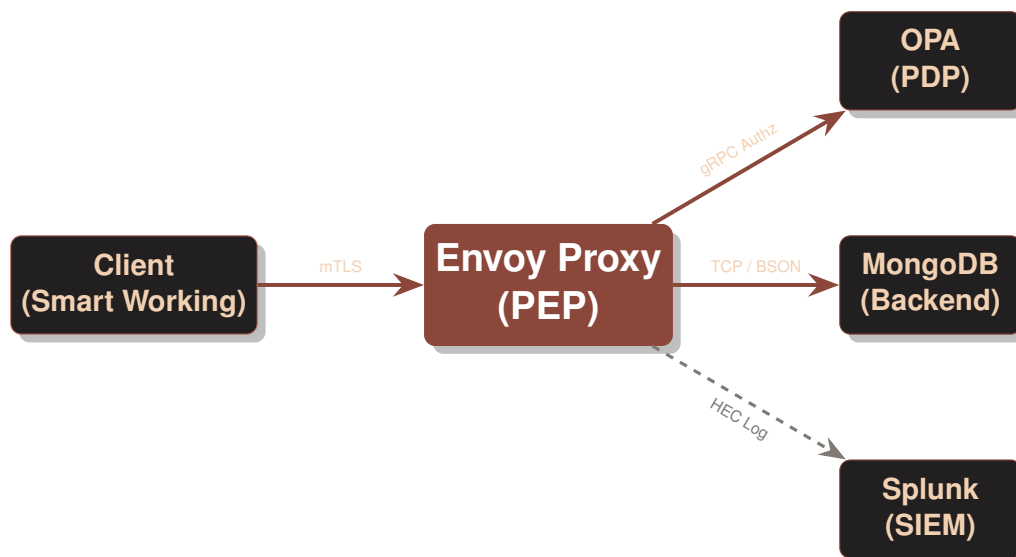


Figura 2: Diagramma dei flussi informativi e dei componenti della Zero Trust Architecture.

Il funzionamento del proxy prevede la separazione dei flussi su porte distinte per gestire in modo sicuro i diversi protocolli. Sulla porta 8443 viene gestito il traffico HTTP cifrato per le Web API aziendali, mentre sulla porta 27017 transita il traffico TCP nativo diretto al database, anch'esso incapsulato in una connessione TLS protetta. Per ogni connessione, il proxy avvia una negoziazione crittografica pretendendo la presentazione di un certificato client valido (*mTLS*). Durante questa fase, viene calcolato l'hash della firma del software client per determinare il fingerprinting *JA3*.

Prima di inoltrare la richiesta verso il backend, il proxy sospende la sessione e interroga OPA tramite una chiamata gRPC. Solo dopo aver ricevuto un responso positivo dal PDP, il proxy provvede a stabilire la connessione con la risorsa richiesta nel backend, garantendo l'applicazione del paradigma Zero Trust per ogni singola transazione.

Questa separazione dei ruoli rispecchia una precisa stratificazione a livello della pila protocollare ISO/OSI. Mentre il firewall *nftables* opera a livello di rete e trasporto (L3/L4), intercettando e filtrando i pacchetti in base agli indirizzi IP e alle porte TCP/UDP, il proxy *envoy* lavora fino al livello applicativo (L7) e di sessione/presentazione (TLS), consentendo una validazione profonda e contestuale che non sarebbe possibile con un tradizionale filtro di rete a pacchetti.

2.4 La tupla multidimensionale Zero Trust (ZTA 6D)

Il nucleo del motore decisionale risiede nella formalizzazione matematica e logica del contesto di accesso attraverso una tupla a sei dimensioni, definita come:

$$T = (u, d, s, n, a, r) \quad (1)$$

Ciascuna dimensione mappa un attributo specifico estratto dinamicamente a livello L4-L7 e valutato in tempo reale per determinare l'affidabilità della richiesta:

1. **Utente** ($u \in U$): l'identità forte del soggetto richiedente, estratta dal campo Subject del certificato client X.509 presentato durante la fase di handshake mTLS (DOWNSTREAM_PEER_SUBJECT). Questa dimensione identifica univocamente gli operatori (*Alice*, *Bob*, *Charlie*).
2. **Dispositivo** ($d \in D$): lo stato di postura dell'hardware. Viene validato verificando la presenza di un'estensione X.509 personalizzata con Object Identifier (OID) specifico 1.3.6.1.4.1.9999.1. Tale estensione attesta che la chiave privata del client sia custodita all'interno di un chip **TPM (Trusted Platform Module)** hardware o emulato, discriminando tra workstation sicure autorizzate e dispositivi non censiti.
3. **Software** ($s \in S$): il fingerprint crittografico della suite client, calcolato tramite l'algoritmo **JA3**. Questo consente a Envoy di identificare il software effettivo che avvia la connessione (es. un browser autorizzato vs uno script Python malevolo), prevenendo attacchi basati su spoofing dell'User-Agent.
4. **Contesto di Rete** ($n \in N$): la provenienza geografica o topologica del pacchetto, classificata in base all'indirizzo IP di origine (DOWNSTREAM_REMOTE_ADDRESS). Permette di distinguere le richieste provenienti dalla rete locale interna ospedaliera (10.0.1.0/24) da quelle provenienti dall'esterno (Smart Working tramite la subnet 192.168.100.0/24).
5. **Azione** ($a \in A$): l'operazione richiesta. Nel caso del protocollo HTTP corrisponde al metodo (es. GET, POST), mentre per le connessioni database MongoDB viene estratta a livello L7 dal filtro mongo_proxy (es. comandi di find, insert, update o delete).
6. **Risorsa** ($r \in R$): il target dell'azione, rappresentato dall'URI del servizio API HTTP richiesto o dal nome della collezione MongoDB (es. patients, medical_records) estratta dinamicamente tramite ispezione profonda dei pacchetti (DPI).

2.5 Orchestrazione e Isolamento dei Container

L'intera architettura è stata implementata e ingegnerizzata utilizzando **Docker** e **Docker Compose**, per garantire la massima portabilità e un isolamento rigoroso a livello di sistema operativo. Come mostrato nel `docker-compose.yaml`, l'infrastruttura è composta da diversi container, ognuno con un ruolo specifico e confinato in base al principio del minimo privilegio.

Le reti virtuali agiscono come veri e propri *VLAN tagging* all'interno dello stack Docker. Ad esempio, il container `zta-mongodb` è collegato esclusivamente alla rete

backend-net, rendendolo inaccessibile a chiunque non possieda un'interfaccia sulla stessa rete (come envoy). Analogamente, il zta-firewall espone un'interfaccia su frontend-net, backend-net ed external-net, fungendo da gateway per instradare o dropare i pacchetti ancor prima che raggiungano il livello applicativo.

L'isolamento non è limitato solo alla rete, ma si estende ai namespace e ai privilegi: il container zta-snort (IDS) condivide lo stesso *network namespace* di Envoy (`network_mode: "service:envoy-pep"`), permettendogli di intercettare passivamente tutto il traffico che transita per il proxy, senza però avere la possibilità di interferire o bloccare direttamente la comunicazione.

2.6 Il server database di risorsa (MongoDB)

Il nucleo di memorizzazione persistente dei dati clinici, denominato *secure core*, è costituito dal database orientato ai documenti mongodb, ospitato nel container zta-mongodb e isolato interamente all'interno della rete privata backend-net.

Il database aziendale gestisce in primis la collezione *patients*, la quale memorizza le cartelle cliniche dei pazienti. Ciascun documento contiene dati anagrafici e clinici di base come `patient_id`, `name`, `age`, `ward` (reparto di ricovero), `blood_type` (gruppo sanguigno) e informazioni altamente confidenziali come il piano terapeutico. Nelle iterazioni iniziali del progetto, lo stato di fiducia dei dispositivi veniva memorizzato staticamente all'interno del database (tramite collezioni dedicate allo ZTA Policy Store); nell'architettura definitiva, tale approccio è stato superato demandando la valutazione della *trust* interamente ai modelli predittivi del SIEM (splunk MLTK). Questa scelta trasforma il database in un puro contenitore di dati operativi, rispettando pienamente il principio di separazione delle responsabilità (*Separation of Concerns*).

Dal punto di vista dell'accesso di rete, il container del database non espone alcuna porta verso l'esterno né è direttamente raggiungibile dai client. Qualsiasi interrogazione diretta deve transitare obbligatoriamente attraverso il modulo proxy envoy, che intercetta le chiamate sulla porta standard 27017. Sfruttando il filtro di rete specifico per MongoDB (`envoy.filters.network.mongo_proxy`), il proxy decodifica il protocollo di comunicazione (inclusi i moderni OP_MSG) ed estrae il payload delle query. Ciò consente ad OPA di eseguire l'enforcement tramite *Deep Packet Inspection* (L7 DPI), applicando politiche di blocco istantaneo su query distruttive (es. `dropDatabase` o cancellazioni massive), proteggendo l'integrità del dato ancor prima che la richiesta raggiunga il motore di storage. L'accesso alle informazioni cliniche sensibili viene invece filtrato a monte sui rispettivi endpoint delle Web API HTTP.

2.7 Le Web API e il portale client di simulazione

Il frontend dell'applicazione e la logica di business sono stati ingegnerizzati per simulare l'interazione quotidiana in un ambiente ospedaliero reale e validare l'efficacia del perimetro Zero Trust.

Il backend applicativo è implementato in Python tramite il framework FastAPI nel container zta-web-api. Oltre a fungere da ponte verso il database MongoDB per l'estrazione delle cartelle cliniche tramite gli endpoint `/api/patients` e `/api/patients/sensitive`, il servizio assume formalmente il ruolo di **Policy Information Point (PIP)** all'interno del framework ABAC. Ricevuta la query formattata da OPA (contenente le dimensioni contestuali *ZTA 6D*), il PIP arricchisce dinamicamente

il *payload* iniettandovi le variabili comportamentali dell'utente (es. `failed_logins`, `session_freq`). La scelta ingegneristica di demandare il tracciamento di tale stato comportamentale all'API, anziché calcolarlo nativamente all'interno di Splunk tramite query di aggregazione sui log storici, risponde a precise esigenze di ottimizzazione architetturale. In un ambiente di produzione prototipale, infatti, l'interrogazione continua di indici di log per ogni singola transazione introdurrebbe una latenza incompatibile con i severi vincoli temporali della *Continuous Authorization*. Di conseguenza, l'API funge da strato di mediazione ad altissima velocità: gestisce le metriche comportamentali e le fornisce pre-elaborate all'endpoint di Splunk MLTK, il quale viene interrogato esclusivamente nella sua veste di motore di inferenza matematica per ottenere la predizione istantanea del rischio, restituita poi al motore decisionale.

Il portale web a cui accedono i medici è erogato in modo indipendente per ciascun client simulato (`client-alice`, `client-bob`, `client-charlie`) tramite container dedicati che eseguono un server web locale (`simulator.py`) e presentano interfacce in HTML/CSS moderno. I tre client simulano diverse posture di sicurezza. L'utente Alice rappresenta un medico operante da una workstation d'ufficio interna autorizzata, dotata di chip TPM e attestazione hardware valida sulla subnet `10.0.1.0/24`. Alice accede correttamente al portale, potendo consultare senza limitazioni sia i dati clinici standard che quelli sensibili. Il client di Bob descrive l'accesso tramite un dispositivo sprovvisto di attestazione crittografica TPM: OPA intercetta la mancanza del requisito hardware e blocca la sessione fin dal form di login (errore 403), applicando il principio di *Short-Circuiting* per evitare chiamate ridondanti al SIEM. Infine, il client di Charlie simula un operatore in modalità *smart working* connesso da una rete esterna (`192.168.100.0/24`). Pur essendo fuori dal perimetro aziendale, Charlie dispone di un dispositivo sicuro con TPM; il sistema gli concede quindi l'accesso di base al portale, avendo un rischio inferiore alla soglia ristretta per gli utenti remoti (soglia ≤ 26).

Per convalidare sul campo le politiche impostate, il sistema implementa un approccio di *Defense in Depth* (Difesa in Profondità) per contrastare gli attacchi. Gli attacchi di livello applicativo (L7) tramite le Web API vengono intercettati in prima linea dal PEP: la barra di ricerca del portale invia input contenenti parole chiave distruttive (es. `DROP` o `DELETE`) che generano una richiesta HTTP anomala, la quale viene ispezionata e bloccata istantaneamente dal filtro Lua nativo di Envoy. Per il traffico diretto verso il database, è invece OPA a eseguire la *Deep Packet Inspection* (DPI) sui payload estratti dal proxy sulla porta `27017`. Le regole di autorizzazione continua di OPA intervengono anche per bloccare utenti legittimi ma contestualmente a rischio (es. Charlie su rete esterna bloccato sulla rotta `/api/patients/sensitive`). Infine, i tentativi di bypass a livello L4 eseguiti tramite terminale client (es. comandi `curl` diretti) vengono scartati dal firewall `nftables`, mentre il sistema IDS Snort notifica l'evento anomalo a Splunk.

CAPITOLO 3

Implementazione pratica e configurazione

In questo capitolo viene analizzata l'implementazione pratica dell'architettura Zero Trust, dettagliando le configurazioni dei singoli container e i meccanismi di interazione tra i moduli di controllo e protezione del traffico.

3.1 La microsegmentazione e il deploy con docker-compose

Come già discusso, il deploy dell'infrastruttura è orchestrato tramite la tecnologia Docker, definendo quattro reti virtuali distinte e isolate nel file `docker-compose.yaml`. Questa separazione impedisce il routing diretto tra i client esterni o di frontend e il secure core di backend, forzando il transito del traffico attraverso il proxy Envoy. per chiarezza riprendiamo la configurazione delle reti isolate definita nel file mostrato nel Listato 2.

```
docker-compose.yaml

1 networks:
2   frontend-net:
3     driver: bridge
4     ipam:
5       config:
6         - subnet: 10.0.1.0/24
7   backend-net:
8     driver: bridge
9   control-plane-net:
10    driver: bridge
11   external-net:
12    driver: bridge
13    ipam:
14      config:
15        - subnet: 192.168.100.0/24
```

Listing 2: Definizione delle quattro reti segregate nel file `docker-compose.yaml`.

3.2 La configurazione di Envoy Proxy come PEP

Il proxy envoy adempie al ruolo fondamentale di **Policy Enforcement Point (PEP)** all'interno dell'architettura progettata, operando come un reverse proxy trasparente e sicuro incaricato di intercettare, decifrare e ispezionare la totalità del traffico dati diretto verso il *secure core*. La configurazione, interamente definita nel file `envoy.yaml`, implementa una rigorosa segregazione dei flussi mediante la dichiarazione di due listener principali operanti su porte distinte, scongiurando qualsiasi possibilità di bypass dei controlli:

- **Listener database (mongodb_listener):** istanziato sulla porta standard 27017, gestisce il traffico TCP nativo verso il database MongoDB. Il blocco `transport_socket` impone l'obbligatorietà del protocollo mutuo TLS (*mTLS*) configurando il parametro `require_client_certificate: true`, costringendo ogni client a presentare un certificato x509 valido per l'attestazione hardware.
- **Listener HTTP (http_listener):** operativo sulla porta 8443, intercetta le chiamate in ingresso dirette alle Web API ospedaliere, decodificando il protocollo HTTP/1.1 o HTTP/2 ed esponendo i servizi web protetti da cifratura end-to-end.

Al fine di estrarre in tempo reale le metriche contestuali (*ZTA 6D*) necessarie al calcolo del rischio comportamentale e abilitare i meccanismi di *Deep Packet Inspection* (DPI), Envoy si avvale di una struttura sequenziale nota come **Filter Chain** (Catena dei Filtri).

Nel listener del database, il flusso informativo attraversa tre moduli specializzati. Inizialmente, il filtro di rete `tls_inspector` interviene durante l'handshake TLS iniziale analizzando il pacchetto di `Client Hello` prima della decifrazione, calcolando l'hash crittografico delle suite di cifratura per ricavare il fingerprinting *JA3*. Successivamente, il modulo applicativo `mongo_proxy` decodifica il protocollo binario BSON (inclusi i moderni opcode `OP_MSG`), estraendo dinamicamente i metadati associati al comando eseguito e alla collezione target. Infine, il filtro `ext_authz` sospende l'elaborazione del socket e inoltra in modo sincrono un payload gRPC (HTTP/2) al server di autorizzazione esterno OPA, includendo sia il certificato client in formato PEM (per la validazione del modulo TPM) sia i metadati estratti da MongoDB. Un estratto dettagliato di tale architettura di rete è documentato nel Listato 3.

```

1 static_resources:
2   listeners:
3     - name: mongodb_listener
4       address:
5         socket_address:
6           address: 0.0.0.0
7           port_value: 27017
8       listener_filters:
9         - name: envoy.filters.listener.tls_inspector
10          typed_config:
11            "@type": type.googleapis.com/envoy.extensions.filters.
12              listener.tls_inspector.v3.TlsInspector
13            enable_ja3_fingerprinting: true
14          filter_chains:
15            - filters:
16              - name: envoy.filters.network.mongo_proxy
17                typed_config:
18                  "@type": type.googleapis.com/envoy.extensions.filters.
19                    network.mongo_proxy.v3.MongoProxy
20                  stat_prefix: mongodb
21                  emit_dynamic_metadata: true
22              - name: envoy.filters.network.ext_authz
23                typed_config:
24                  "@type": type.googleapis.com/envoy.extensions.filters.
25                    network.ext_authz.v3.ExtAuthz
26                  stat_prefix: opa_authz
27                  transport_api_version: V3
28                  grpc_service:
29                    envoy_grpc:
30                      cluster_name: opa_cluster
31                  timeout: 45s
32                  include_peer_certificate: true

```

Listing 3: Estratto della configurazione del listener MongoDB in envoy.yaml.

Per quanto concerne il listener HTTP protetto sulla porta 8443, la catena dei filtri applicativi (gestita dall'HttpConnectionManager) implementa un approccio strategico di **Defense in Depth** (Difesa in Profondità). Prima che la richiesta sia trasmessa al backend o sottoposta alla valutazione del contesto da parte di OPA, intervengono due filtri locali in linea. Il modulo `header_mutation` inietta proattivamente l'hash JA3 estratto all'interno dell'header HTTP personalizzato `x-client-fingerprint`.

Immediatamente dopo, interviene un firewall L7 programmabile basato sul motore lua (`envoy.filters.http.lua`). Tale componente agisce come un filtro stateless ad altissima velocità, incaricato di validare la struttura formale della richiesta. Come evidenziato nel codice reale riportato nel Listato 4, lo script intercetta preventivamente i tentativi di attacco basati su metodi HTTP distruttivi (es. chiamate di tipo DELETE dirette agli endpoint dei pazienti), respingendoli immediatamente con un codice di errore 403 Forbidden. Questa stratificazione consente di sgravare il PDP (OPA) e i modelli di Machine Learning di Splunk dal computo di richieste palesemente malevole, ottimizzando la latenza complessiva del sistema. Le logiche di autorizzazione contestuale granulari

— come la limitazione d’accesso alla rotta sensibile per gli utenti in smart working — vengono invece delegate al successivo modulo `ext_authz`, preservando la separazione tra filtri strutturali e decisioni adattive.

envoy.yaml (Filtro HTTP Lua)

```

1      - name: envoy.filters.http.lua
2        typed_config:
3          "@type": type.googleapis.com/envoy.extensions.filters.
4            http.lua.v3.Lua
5          inline_code: |
6            function envoy_on_request(request_handle)
7              local path = request_handle:headers():get(":path")
8              local method = request_handle:headers():get(":
9                method")
10
11              -- L7 DPI Firewall Rules
12              if method == "DELETE" and string.match(path, "^/api
13                /patients") then
14                request_handle:respond({[":status"] = "403"}, '{"
15                  error": "L7 Firewall Block: Destructive operations (DROP) are
16                    forbidden"}')
17              end
18            end

```

Listing 4: Configurazione del filtro HTTP Lua in `envoy.yaml` per la prima linea di difesa L7.

3.3 Formalizzazione del Controllo degli Accessi: Modello ABAC e Matrice Dinamica

Il perimetro di sicurezza progettato supera il concetto statico di autorizzazione per implementare un modello dinamico riconducibile all’architettura **ABAC** (*Attribute-Based Access Control*). In accordo con la teoria del controllo degli accessi e il modello di Harrison, Ruzzo e Ullman (HRU), il dominio di protezione è definito da tre insiemi fondamentali:

- L’insieme dei **Soggetti** $S = \{s_1, \dots, s_n\}$, che rappresentano le entità computazionali richiedenti (es. gli operatori ospedalieri o i microservizi).
- L’insieme degli **Oggetti** $O = \{o_1, \dots, o_m\}$, che costituiscono le risorse da proteggere (es. endpoint HTTP, database, cartelle cliniche).
- L’insieme dei **Diritti** $R = \{r_1, \dots, r_k\}$, che definiscono le operazioni eseguibili (es. GET, POST, find, drop).

Nel modello HRU classico, il sistema dei permessi è rappresentato da una **Matrice di Access Control** bidimensionale A , dove ciascuna voce $A[s_i, o_j] \subseteq R$ indica staticamente i diritti che il soggetto s_i possiede sull’oggetto o_j . L’operazione r è consentita se e solo se $r \in A[s_i, o_j]$.

Nell’architettura Zero Trust sviluppata, tale approccio discrezionale e statico viene superato in favore del paradigma ABAC: le identità e il contesto di rete non fungono

più da indici statici per la matrice, ma vengono tradotti in un *insieme di attributi*. Come definito dalla teoria, nell'ABAC i permessi non sono semplici associazioni preconfigurate, bensì *espressioni booleane (condizioni) valutate dinamicamente sugli attributi* del richiedente.

Di conseguenza, la cella della matrice $A[s_i, o_j]$ cessa di essere una locazione di memoria fissa per trasformarsi in una funzione di autorizzazione calcolata in tempo reale dal Policy Decision Point (OPA), dipendente dallo stato corrente degli attributi:

$$A[s_i, o_j] = \{r \in R \mid D(s_i, o_j, r, \text{Attributi}) = \text{True}\}$$

Le *Corporate Security Policy* sono state formalizzate in OPA traducendo le seguenti proposizioni logiche inderogabili sui flussi di informazione:

Policy 1: Requisito Hardware di Autenticazione (Short-Circuit)

Nessun diritto di accesso iniziale può essere concesso in assenza di una radice di fiducia hardware verificata. Sia $s_{tpm} \in \{0, 1\}$ l'attributo che certifica la presenza del chip TPM:

$$\forall o \in \text{Login}, \quad \text{se } s_{tpm} = 0 \implies A[s, o] = \emptyset$$

Policy 2: Autorizzazione Adattiva (Maximum e Low Trust)

L'assegnazione dei diritti dipende dal punteggio di rischio $s_{risk} \in [0, 100]$ calcolato algoritmamente (Splunk MLTK) e dalla provenienza di rete s_{net} . Per un utente interno con TPM, la matrice abilita i diritti finché il rischio si mantiene entro l'intervallo di confidenza statistica $(\mu + 3\sigma)$.

$$\text{se } s_{net} = \text{Internal} \wedge s_{tpm} = 1 \wedge s_{risk} \leq 38 \implies r \in A[s, o]$$

Al contrario, per scenari di Smart Working ($s_{net} = \text{External}$), si impone il principio del guinzaglio corto $(\mu + 1.5\sigma)$:

$$\text{se } s_{net} = \text{External} \wedge s_{tpm} = 1 \wedge s_{risk} \leq 26 \implies r \in A[s, o]$$

Policy 3: Confidenzialità e Segregazione del Dato Sensibile (DPI Livello 1)

In accordo con le politiche di flusso delle informazioni (impedire letture non autorizzate verso domini non fidati), l'accesso alle risorse altamente sensibili (o_{sens}) è vincolato a requisiti stringenti. Se l'utente opera da rete esterna, la cella della matrice viene svuotata dinamicamente:

$$\text{se } o = o_{sens} \wedge (s_{net} \neq \text{Internal} \vee s_{tpm} = 0) \implies \text{GET} \notin A[s, o_{sens}]$$

Policy 4: Integrità e Protezione Attiva (DPI Livello 2)

Qualsiasi diritto classificato come distruttivo ($r \in \{\text{drop}, \text{deleteall}\}$) non può mai essere concesso all'interno del flusso applicativo, garantendo la disponibilità e l'integrità del dato clinico:

$$\forall s \in S, \forall o \in O, \quad \{\text{drop}, \text{deleteall}\} \notin A[s, o]$$

In sintesi, la matrice degli accessi A muta la propria topologia adattandosi ad ogni transazione. La cella che relaziona l'utente in smart working alla cartella clinica

conterrà il diritto di lettura GET fintantoché il rischio comportamentale sarà accettabile, ma subirà la revoca istantanea (la cella diverrà l'insieme vuoto $\emptyset$) non appena si manifesterà un'anomalia nel SIEM o una disconnessione dalla rete sicura.

3.4 Il motore decisionale OPA (PDP) e l'integrazione Splunk MLTK

Il Policy Decision Point (PDP) riceve la richiesta di autorizzazione da Envoy ed esegue la valutazione delle policy definite in linguaggio Rego all'interno del file `rules.rego`. Il cuore del sistema è progettato per supportare la *Continuous Authorization* basata sul contesto, implementando quello che in letteratura è definito un **Contextual and Score-based Trust Algorithm**. Per fare ciò, OPA interroga sincronicamente l'API predittiva di Splunk MLTK per ottenere una valutazione istantanea della fiducia basata sulla storia recente e sul comportamento dell'utente.

La query predittiva, strutturata nel linguaggio SPL (*Search Processing Language*) di Splunk come mostrato nel Listato 5, viene interpolata dinamicamente da OPA inserendo le sei dimensioni correnti (*ZTA 6D*) estratte dai metadati della connessione.

●●● Splunk SPL Query (Interpolata dinamicamente da OPA)

```
1 | makeresults
2 | eval user="Alice", software="JA3_HASH", device="Workstation
   Ospedaliera Sicura (TPM Validato)", network="10.0.1.5", action="
   find", resource="patients"
3 | apply trust_model
4 | rename "predicted(rischio)" as rischio
5 | table rischio
```

Listing 5: Struttura della query Splunk MLTK generata dinamicamente da OPA.

Per ottimizzare le prestazioni architetturali e prevenire la saturazione del SIEM con traffico evidentemente malevolo, la chiamata al modello di Machine Learning è preceduta da una **clausola di guardia** (*Short-Circuiting*) che agisce come un **Criteria-based Trust Algorithm**. Se una richiesta non soddisfa i criteri infrastrutturali di base (ad esempio, un tentativo di login senza attestazione TPM), OPA interrompe la valutazione, assegna il rischio massimo di default e rifiuta la connessione senza interpellare Splunk.

Accanto all'analisi comportamentale, OPA implementa un motore di *Deep Packet Inspection* (L7 DPI) strutturato su due livelli di difesa per garantire la corretta imposizione degli obiettivi di **Confidenzialità** e **Integrità** sui flussi di informazione, come illustrato nel Listato 6. Il primo livello (Confidenzialità) interviene sul traffico HTTP per segregare l'accesso ai dati sensibili dei pazienti (`/api/patients/sensitive`), bloccando preventivamente i flussi verso domini non fidati se l'utente proviene da una rete esterna o è sprovvisto di modulo TPM. Il secondo livello (Integrità) protegge il database decodificando i payload nativi BSON ed inibendo l'esecuzione di query distruttive dirette (come `dropdatabase`), prevenendo alterazioni di stato non autorizzate.

rules.rego (DPI L7 e Protezione Sensibile)

```

1 # L7 DPI: Regole di blocco esplicite su traffico HTTP e MongoDB
2 default 17_dpi_block := false
3
4 # --- LIVELLO 1: Policy di Confidenzialita (Endpoint HTTP Sensibili)
5   ---
6 17_dpi_block := true if {
7     is_http
8     resource == "/api/patients/sensitive"
9     not is_internal_network # Blocca reti esterne
10 } else := true if {
11     is_http
12     resource == "/api/patients/sensitive"
13     not is_tpm # Blocca assenza TPM
14 }
15
16 # --- LIVELLO 2: Policy di Integrita (Comandi nativi MongoDB) ---
17 else := true if {
18     is_mongo
19     contains(lower(db_query), "dropdatabase")
20 } else := true if {
21     is_mongo
22     contains(lower(db_query), "deleteall")
23 }

```

Listing 6: Regole di ispezione profonda L7 DPI per confidenzialità e integrità in rules.rego.

Il verdetto finale di accesso (allow) viene deliberato calcolando un livello di confidenza basato sullo stato dell'hardware, la localizzazione di rete e il punteggio dinamico di Splunk (splunk_risk_score), come dettagliato nel Listato 7. Le soglie di tolleranza deterministiche implementate (26 e 38) non sono arbitrarie, ma derivano rigorosamente dall'analisi statistica della distribuzione normale (*curva di Gauss*) dei log storici legittimi elaborati dall'algoritmo del SIEM.

Nello specifico, il valore di 38 rappresenta la soglia di tolleranza massima assegnata a un utente interno ideale dotato di attestazione hardware (*Alice*). Tale barriera viene calcolata impostando un intervallo di confidenza pari a tre deviazioni standard oltre la media del comportamento di baseline ($\mu + 3\sigma$), una metrica statistica che scherma il 99.7% delle fluttuazioni operative ordinarie riducendo a zero i falsi positivi, ma intercettando tempestivamente anomalie critiche dovute a sessioni compromesse.

Al contrario, la soglia ridotta a 26 risponde al principio Zero Trust del **guinzaglio corto** (*Low Trust Tolerance*), applicato agli scenari intrinsecamente esposti a maggior rischio perimetrale: gli utenti legittimi in smart working (*Charlie*) e i dispositivi legacy privi di TPM operanti tramite fallback su fingerprinting JA3. Poiché l'algoritmo di generazione del dataset impone a chiunque operi da rete esterna una penalità infrastrutturale fissa di partenza pari a 18, la soglia di 26 limita il margine comportamentale consentito a sole 1.5 deviazioni standard oltre la media ($\mu + 1.5\sigma$). Questo garantisce che una minima deviazione dalla routine quotidiana provochi l'immediato superamento della barriera e il conseguente blocco della transazione da parte di OPA. Infine, per l'autenticazione iniziale (is_auth_request), viene meno qualsiasi approccio *Score-*

based adattivo e si impone l'obbligo *Criteria-based* statico e vincolante del modulo TPM, neutralizzando gli attacchi brute-force prima ancora della valutazione del rischio.

●●● rules.rego (Regole di autorizzazione adattiva)

```

1 # La regola di autorizzazione adattiva (Risk-Based & JA3 Fallback)
2 risk_ok := true if {
3     is_internal_network
4     is_tpm
5     splunk_risk_score <= 38
6 } else := true if {
7     is_internal_network
8     not is_tpm
9     software != "Sconosciuto"
10    splunk_risk_score <= 26
11 } else := true if {
12     not is_internal_network
13     is_tpm
14     splunk_risk_score <= 26
15 } else := false
16
17 # Blocca preventivamente l'autenticazione senza TPM
18 is_auth_blocked := true if {
19     is_auth_request
20     software == "Sconosciuto"
21 } else := true if {
22     is_auth_request
23     not is_tpm
24 } else := false
# <--- BLOCCO TASSATIVO LOGIN SENZA TPM

```

Listing 7: Regole di autorizzazione adattiva (Score-based) e short-circuiting (Criteria-based) in rules.rego.

3.5 Regole di network protection e tracciamento su nftables

Il componente firewall, basato sul framework nftables del kernel Linux, agisce come barriera L3/L4 di frontiera. Esso blinda l'architettura forzando l'instradamento obbligatorio verso il *Policy Enforcement Point* (Envoy) e gestendo una trappola attiva per rilevare e stroncare i tentativi di bypass della crittografia o del proxy.

All'avvio del container tramite lo script `start.sh`, il firewall esegue quattro task architetturali primari:

1. **Risoluzione dinamica:** interroga il resolver DNS interno di Docker per ottenere l'IP assegnato dinamicamente al container del proxy (zta-envoy).
2. **Segregazione e NAT (Network Address Translation):** reindirizza tutto il traffico TCP in ingresso verso l'IP di Envoy sulla porta sicura 8443 tramite una regola di *Destination NAT* (DNAT) operante nella catena di prerouting, applicando contestualmente il mascheramento SNAT per garantire la corretta commutazione dei pacchetti di ritorno.

3. **Sinkhole e Trappola di bypass (L4):** intercetta qualsiasi tentativo di connessione diretta (spesso sintomo di movimento laterale) alle porte di backend non autorizzate (come la porta 8000 delle Web API). Il traffico anomalo genera prima un log a livello kernel verso il demone `ulogd2` con il prefisso `[NFT-BLOCK]`, per poi essere scartato istantaneamente (drop).
4. **Automated Remediation (Denylist dinamica):** istanzia un set denominato `denylist` per il blocco assoluto (forward drop) degli indirizzi IP compromessi, la cui popolazione avviene in tempo reale.

La configurazione delle catene, dei set e delle regole di NAT applicate all'avvio del firewall è mostrata nel Listato 8.

●●● **start.sh (Configurazione nftables)**

```

1 # --- Inizializzazione Base NFTables ---
2 nft flush ruleset
3 nft add table ip filter
4 nft add set ip filter denylist { type ipv4_addr \; }
5 nft add chain ip filter forward { type filter hook forward priority 0
   \; policy accept \; }
6
7 # Regole di DROP assoluto per gli IP bannati
8 nft add rule ip filter forward ip saddr @denylist counter drop
9
10 # --- Port Forwarding (DNAT) verso Envoy (PEP) ---
11 nft add table ip nat
12 nft add chain ip nat prerouting { type nat hook prerouting priority
   -100 \; }
13 nft add chain ip nat postrouting { type nat hook postrouting priority
   100 \; }
14 nft add rule ip nat prerouting tcp dport 8443 counter dnat to
   $ENVOY_IP:8443
15 nft add rule ip nat postrouting ip daddr $ENVOY_IP tcp dport 8443
   counter masquerade
16
17 # --- TRAPPOLA: Log e Drop del traffico L4 diretto al backend ---
18 nft add rule ip nat prerouting tcp dport 8000 log group 0 prefix "[
   NFT-BLOCK] " counter
19 nft add rule ip nat prerouting tcp dport 8000 drop

```

Listing 8: Configurazione delle catene di filtering e NAT nello script di avvio del firewall.

Per abilitare l'inserimento dinamico in tale `denylist` senza la necessità di un intervento umano, il container del firewall ospita parallelamente un'API RESTful di management sviluppata in Python tramite il framework `FastAPI`. Questa interfaccia espone l'endpoint `POST /ban`, il quale riceve un indirizzo IP e invoca il comando di sistema per iniettarlo direttamente nel set `denylist` del kernel, fornendo un meccanismo di reazione istantanea utilizzabile da sistemi esterni (come l'IDS `Snort`) in caso di rilevamento di traffico ostile.

Inoltre, il codice Python è strutturato nativamente per operare in sinergia con la pipeline di telemetria dell'architettura. Come definito nella libreria di logging, ogni richiesta di ban o intercettazione genera stringhe di log in formato chiave-valore (ad

esempio `event="nftables_drop" status="added" client_ip="192.168.100.X"`). Questo formato agevola il parsing asincrono da parte del raccoglitore Fluent Bit, garantendo un'ingestion rapida e una visualizzazione strutturata all'interno delle dashboard del SIEM Splunk.

3.6 Autenticazione crittografica: mTLS, TPM e fingerprinting JA3

L'architettura implementa un meccanismo di verifica dell'identità a tre livelli complementari, che operano in modo congiunto durante la fase di handshake TLS per costruire le dimensioni *u* (utente), *d* (dispositivo) e *s* (software) della tupla contestuale Zero Trust.

3.6.1 Mutua autenticazione TLS (mTLS)

Ogni listener di Envoy impone la presentazione obbligatoria di un certificato client X.509 firmato dalla CA interna dell'ospedale (ZTA Hospital Trust Root CA). La direttiva `require_client_certificate: true` nella configurazione TLS di Envoy garantisce che nessuna connessione priva di certificato valido possa superare la fase di negoziazione. Il campo `Common Name (CN)` del certificato identifica univocamente l'operatore (es. `employee-alice`, `employee-bob`) e viene estratto da Envoy per essere propagato al motore OPA attraverso l'attributo gRPC `source.principal` della richiesta `ExtAuthz`.

3.6.2 Attestazione hardware TPM

Per discriminare i dispositivi aziendali sicuri dai terminali personali o compromessi, il sistema utilizza un'estensione X.509 personalizzata con `Object Identifier (OID)` `1.3.6.1.4.1.9999.1`. Ai fini della presente implementazione prototipale, lo script `generate_identities.py` simula il processo di *hardware attestation* arricchendo dinamicamente i certificati destinati alle workstation sicure (come quelle di Alice e Charlie) con tale estensione. Il frammento di generazione è mostrato nel Listato 9.

 generate_identities.py

```

1 TPM_OID = ObjectIdentifier("1.3.6.1.4.1.9999.1")
2
3 def generate_leaf(base_dir, name, cn, ca_cert, ca_key, has_tpm=False)
4     :
5     key = rsa.generate_private_key(public_exponent=65537, key_size
6     =2048)
7     subject = x509.Name([x509.NameAttribute(NameOID.COMMON_NAME, cn)
8     ])
9     builder = x509.CertificateBuilder()
10    # ... (configurazione base del certificato)
11    if has_tpm:
12        builder = builder.add_extension(
13            x509.UnrecognizedExtension(
14                TPM_OID,
15                b"TPM-VERIFIED-HARDWARE-ROOT-OF-TRUST-PCR0-OK"
16            ), critical=False
17        )
18    cert = builder.sign(ca_key, hashes.SHA256())

```

Listing 9: Generazione dei certificati client con estensione TPM simulata.

Nel mondo reale in un ambiente di produzione aziendale, tale estensione verrebbe apposta dalla CA solo dopo aver crittograficamente validato una *Certificate Signing Request* (CSR) firmata direttamente dalla Endorsement Key (EK) di un chip TPM fisico. Questo approccio ingegneristico garantisce la proprietà di inesportabilità della chiave privata del client: essendo generata in hardware, la chiave non può essere esfiltrata e il certificato utente risulta indissolubilmente legato alla macchina autorizzata. A livello di policy, OPA valida questa garanzia di postura in tempo reale decodificando il certificato PEM e verificando la presenza dell'OID specifico nell'array delle estensioni (Listato 10).

 rules.rego

```

1 # C. DISPOSITIVO E TPM
2 default is_tpm := false
3 is_tpm := true if {
4     cert_raw := input.attributes.source.certificate
5     cert_pem := urlquery.decode(cert_raw)
6     certs := crypto.x509.parse_certificates(cert_pem)
7     count(certs) > 0
8     some ext in certs[0].Extensions
9     ext.Id == [1, 3, 6, 1, 4, 1, 9999, 1]
10 }
11 default device := "Dispositivo non censito (No TPM)"
12 device := "Workstation Ospedaliera Sicura (TPM Validato)" if { is_tpm
13 }

```

Listing 10: Validazione dell'estensione TPM nel certificato client da parte di OPA.

3.6.3 Fingerprinting software JA3

Il terzo livello di identificazione opera a livello di trasporto, in maniera del tutto passiva rispetto al client. Durante l'handshake TLS, il filtro `tls_inspector` di Envoy calcola l'hash **JA3**, un'impronta digitale deterministica basata sulla combinazione dei parametri proposti nel pacchetto di `ClientHello` (versione TLS, cipher suite offerte, estensioni, curve ellittiche e formati dei punti). Poiché ogni implementazione software genera un `ClientHello` caratteristico, l'hash JA3 consente di distinguere un browser autorizzato (es. Firefox o Chrome) da uno strumento per l'enumerazione o l'attacco (es. `curl` o `nmap`), neutralizzando le tecniche di spoofing dell'header `User-Agent`. L'impronta calcolata viene propagata ad OPA tramite l'header `x-client-fingerprint` (su protocollo HTTP) o tramite i metadati dinamici di Envoy (su protocollo MongoDB).

Tuttavia, l'impiego del solo fingerprinting software presenta un limite strutturale intrinseco: qualora due dispositivi distinti utilizzino il medesimo browser ed eseguano la stessa versione di sistema operativo, produrranno un hash JA3 perfettamente identico. In questo scenario, i dispositivi risultano crittograficamente indistinguibili. Proprio alla luce di questa vulnerabilità, il motore decisionale Zero Trust classifica il JA3 come indicatore di rischio debole: non lo utilizza mai per garantire l'accesso critico (come la richiesta di login, che fallisce sistematicamente in assenza di TPM hardware), ma lo sfrutta esclusivamente come meccanismo di *fallback* per applicare il guinzaglio corto (soglia ≤ 26) su operazioni operative di basso livello in attesa dell'aggiornamento dei client legacy.

3.7 Rilevamento delle intrusioni di rete: Snort NIDS

A completamento delle difese applicative gestite dal proxy e dal motore OPA, l'architettura integra `snort` come **Network Intrusion Detection System** (NIDS) passivo, posizionato strategicamente in modo da intercettare tutto il traffico perimetrale.

La tecnica di deploy adottata è quella della condivisione del *network namespace*: nel file `docker-compose.yaml`, il container `zta-snort` è configurato con la direttiva `network_mode: "service:envoy-pep"`. Questo approccio di ingegneria di rete consente all'IDS di agganciarsi direttamente all'interfaccia virtuale di Envoy, osservando passivamente tutti i pacchetti in ingresso e in uscita senza possedere un proprio indirizzo IP e senza introdurre latenza, agendo come una vera e propria sonda invisibile (TAP di rete).

All'avvio, Snort viene lanciato in ascolto sull'interfaccia principale del namespace condiviso, riversando l'output dei log in formato compatibile nella directory `/var/log/snort`, montata come volume Docker per l'ingestion asincrona da parte di Fluent-Bit. Le regole personalizzate, definite nel file `local.rules`, superano il tradizionale rilevamento statico per intercettare anomalie volumetriche, tentativi di elusione crittografica e comportamenti operativi sospetti. L'intero set di firme è riportato nel Listato 11.

 local.rules

```

1 # 1. Rileva una scansione Nmap (SYN Scan) verso Envoy o DB
2 alert tcp any any -> any any (msg:"ZTA ALERT: Rilevata Scansione di
   Rete (Possibile Nmap)"; \
3   flags:S; threshold:~type both, track by_src, count 20, seconds 10;
   sid:1000001; rev:1;)
4
5 # 2. AUDIT: Traccia l'accesso tramite client nativo MongoDB
6 alert tcp any any -> any 27017 (msg:"ZTA AUDIT: Connessione Client
   nativa a MongoDB \
7   rilevata (Accesso non-Web)"; sid:1000002; rev:2;)
8
9 # 3. Rileva attacco volumetrico (Possibile DDoS / SYN Flood verso
   Envoy)
10 alert tcp any any -> any 8443 (msg:"ZTA ALERT: Anomalia volumetrica (
   Possibile DoS \
11   verso il Proxy)"; flags:S; threshold:~type both, track by_src, count
   100, seconds 5; \
12   sid:1000003; rev:1;)
13
14 # 4. Rileva un tentativo di connessione in chiaro (Downgrade Attack
   HTTP)
15 alert tcp any any -> any 8443 (msg:"ZTA ALERT: Tentativo di traffico
   in chiaro su porta \
16   protetta (Downgrade/Misconfig)"; flow:established,to_server;
   content:"GET "; depth:4; \
17   sid:1000004; rev:1;)
18
19 # 5. Rileva tentativi ripetuti di connessione (Possibile Brute-Force
   del mTLS)
20 alert tcp any any -> any 8443 (msg:"ZTA ALERT: Eccessivi tentativi di
   connessione \
21   (Possibile assenza di Certificato Client)"; flags:R; threshold:~type
   both, track by_src, \
22   count 15, seconds 10; sid:1000005; rev:1;)

```

Listing 11: Regole di rilevamento avanzate (Volumetriche, Protocolлари e Crittografiche) per Snort.

Il file di configurazione implementa quattro vettori logici di analisi principali:

- **Attacchi volumetrici e Reconnaissance (Regole 1 e 3):** Sfruttano il costrutto `threshold` per identificare anomalie basate sulla frequenza dei pacchetti di sincronizzazione (SYN). La Regola 1 identifica i pattern tipici degli scanner di rete come Nmap, mentre la Regola 3 monitora burst aggressivi diretti esclusivamente al listener web (8443), mitigando visivamente tentativi di *Denial of Service* (SYN Flood) mirati ad esaurire le risorse del proxy.
- **Anomalie Protocolлари e Downgrade (Regola 4):** Esegue una *Deep Packet Inspection* (DPI) sui primi byte del payload TCP alla ricerca di metodi in chiaro (come la stringa "GET "). Rilevare traffico non decifrato su una porta esplicitamente vincolata al TLS indica un'errata configurazione del client o un malintenzionato che tenta un *Downgrade Attack* forzando l'uso del protocollo HTTP standard.

- **Fallimenti Crittografici (Regola 5):** Poiché Envoy impone rigorosamente la mutua autenticazione (*mTLS*), l'assenza di un certificato client valido o la mancata verifica della CA provocano l'immediato abbattimento della connessione da parte del PEP (che innesca l'invio di pacchetti TCP Reset - RST). Questa regola correla un improvviso accumulo di pacchetti RST a un attaccante che tenta ripetutamente l'accesso senza disporre dell'attestazione hardware o delle chiavi crittografiche richieste.
- **Auditing Operativo (Regola 2):** Il sistema rileva attività legittime ma deviatrici rispetto alla baseline. Monitorando gli accessi sulla porta 27017, Snort genera eventi di Audit quando operatori autorizzati (es. personale di Data Science) si collegano direttamente al database tramite client nativi, aggirando le Web API ma passando regolarmente per i controlli mTLS e OPA del proxy.

3.8 Aggregazione centralizzata dei log: Fluent-Bit

La raccolta e la centralizzazione dei log di telemetria provenienti dai diversi componenti dell'infrastruttura è affidata a *fluent-bit*, un agente di log forwarding leggero e ad alte prestazioni. Fluent-Bit è stato preferito al classico *syslog* per la sua capacità nativa di operare in ambienti containerizzati, supportando il parsing strutturato di log JSON e l'invio diretto verso Splunk tramite il protocollo **HTTP Event Collector** (HEC).

La configurazione, definita nel file *fluent-bit.conf*, implementa una pipeline di raccolta multi-sorgente con cinque canali di input e un'unica destinazione di output, come schematizzato nella Tabella 1.

Tabella 1: Pipeline di raccolta log di Fluent-Bit: sorgenti e tag associati.

Sorgente	Tag	Percorso Volume	Host Iniettato
Snort IDS	snort.alerts	/var/log/snort/alert_json.txt	zta-snort
MongoDB	mongo.audit	/var/log/mongodb/mongod.log	zta-mongodb
Envoy PEP	envoy.access	/var/log/envoy/access.log	zta-envoy
OPA PDP	opa.decisions	/var/log/opa/decision.log	zta-opa
NFTables Firewall	nftables.kernel	/var/log/nftables/firewall.log	zta-firewall

L'architettura sfrutta i **volumi Docker condivisi** come meccanismo di trasporto: ogni container scrive i propri log in un volume dedicato (es. *envoy_logs*, *snort_logs*) e Fluent-Bit li legge in modalità *tail* dal percorso montato. Per i log strutturati di OPA, Fluent-Bit applica una catena di filtri dedicata: un primo filtro *grep* seleziona solo le righe contenenti il marcatore [OPA-PDP], un parser con espressione regolare estrae il payload JSON dalla stringa di log, e un secondo parser deserializza il JSON strutturato contenente la decisione, il risk score e le sei dimensioni ZTA.

Tutti i log vengono infine inoltrati verso l'unica destinazione: il **Splunk SIEM** tramite il modulo di output *splunk* configurato con il token HEC e la connessione TLS verso la porta 8088 del container *splunk-siem*.

3.9 Il modello predittivo di Machine Learning: Splunk MLTK e Gradient Boosting

Il calcolo del punteggio di rischio dinamico rappresenta il motore decisionale della valutazione adattiva continua dell'architettura Zero Trust. L'intelligenza predittiva è implementata all'interno del SIEM tramite lo **Splunk Machine Learning Toolkit** (MLTK), segnando il passaggio da un approccio di autorizzazione statico a un'analisi comportamentale in tempo reale.

3.9.1 Dall'approccio statico alla valutazione comportamentale continua

L'architettura Zero Trust di base si fonda su un'analisi del contesto di autenticazione composta da 6 dimensioni identitarie (*user, software, device, network, action, resource*). Per abilitare la *Continuous Authorization*, il modello integra ulteriori 6 feature comportamentali, iniettate dinamicamente dal proxy Web API. Tali feature trasformano la fotografia istantanea dell'accesso in un monitoraggio continuo della sessione:

- `failed_logins`: tentativi di accesso falliti nelle ultime 24 ore.
- `hour_of_day` e `is_night`: fascia oraria e flag per accessi in orario notturno (22:00–06:00).
- `session_freq`: frequenza delle richieste nell'ultima ora (indicatore anti-bot).
- `sensitivity_level`: grado di criticità della risorsa richiesta (es. dati pazienti vs configurazioni).
- `days_inactive`: giorni trascorsi dall'ultimo login riuscito.

3.9.2 Generazione del dataset e modello matematico

Il modello viene addestrato su un dataset sintetico di 50.000 record. Il valore target predittivo, ovvero il rischio (in scala 0 – 100), è determinato nel dataset di addestramento tramite una formula deterministica non lineare che cumula penalità infrastrutturali e comportamentali:

$$\text{Rischio} = \Delta_{\text{HW}} + \Delta_{\text{Rete}} + (\Delta_{\text{Comp}} \times M_{\text{Sens}}) + \epsilon$$

I parametri riflettono severità architetturali precise, in accordo con la configurazione del PEP (es. $\Delta_{\text{HW}} = +30$ per dispositivi sprovvisti di attestazione TPM, $\Delta_{\text{Rete}} = +18$ per connessioni da reti esterne). Il termine Δ_{Comp} aggrega le anomalie di comportamento, amplificate dal moltiplicatore di sensibilità della risorsa (M_{Sens}). Infine, $\epsilon \sim \mathcal{N}(0, 2)$ introduce un rumore statistico gaussiano, vitale per prevenire l'overfitting e addestrare l'algoritmo a generalizzare.

3.9.3 Selezione dell'algoritmo: Gradient Boosting vs Random Forest

La scelta dell'algoritmo **Gradient Boosting Regressor** in contrapposizione a tecniche parallele (come la *Random Forest*) è motivata dall'intrinseca natura composita del rischio informatico.

Nel paradigma *Random Forest* (basato sul *bagging*), vengono creati molteplici alberi decisionali in parallelo e la predizione finale è la media aritmetica dei risultati.

Sebbene efficace per ridurre la varianza, questa mediazione tende a "smussare" le anomalie, un difetto critico nel dominio della sicurezza dove un picco anomalo non deve essere attenuato. Al contrario, il Gradient Boosting opera secondo il principio dell'**apprendimento additivo sequenziale**. L'algoritmo costruisce un albero alla volta; ogni nuovo *weak learner* viene addestrato specificamente per approssimare i residui (gli errori di stima) commessi dall'albero precedente, minimizzando direttamente la *Loss Function* calcolando il gradiente dell'errore.

Questo approccio risulta superiore nel catturare le relazioni non lineari tra le 12 feature ZTA: un utente che fallisce 5 login di giorno dall'ufficio possiede un profilo di rischio moderato, ma lo stesso utente che fallisce 5 login alle 3 di notte da un IP esterno rappresenta una minaccia critica. Focalizzandosi sulla correzione degli errori sui casi limite, il Gradient Boosting penalizza severamente queste combinazioni atipiche.

3.9.4 Addestramento e Regressione in tempo reale

L'addestramento avviene direttamente nell'interfaccia di Splunk tramite il costruito SPL mostrato nel Listato 12. Una volta generato, il modello viene configurato con permessi *Globali* per consentirne l'interrogazione esterna tramite API REST.

●●● Splunk SPL | Addestramento del Trust Model

```
1 | inputlookup simulated_traffic.csv
2 | fit GradientBoostingRegressor "rischio"
3   from user, software, device, network, action, resource,
4   failed_logins, hour_of_day, is_night,
5   session_freq, sensitivity_level, days_inactive
6   into trust_model
```

Listing 12: Query SPL per l'addestramento del modello Gradient Boosting su Splunk MLTK.

Il vantaggio operativo di questo approccio risiede nell'utilizzo della **Regressione** anziché della Classificazione. Il modello non restituisce un giudizio binario (sicuro/pericolo), ma un punteggio numerico continuo (es. 18.4, 42.7). Durante il funzionamento, OPA ingerisce dinamicamente questo valore per attuare politiche di autorizzazione adattive, applicando le soglie di tolleranza predefinite (es. blocco per rischio > 38 su rete interna o > 26 su rete esterna), rendendo il perimetro di sicurezza fluido e proporzionale alla postura reale dell'utente.

CAPITOLO 4

Simulazione degli scenari e validazione

Per verificare la robustezza dell'infrastruttura Zero Trust progettata, sono stati simulati cinque scenari operativi principali, ognuno mirato a testare specifiche componenti difensive e logiche del sistema. Tutti gli utenti ed operatori si interfacciano inizialmente con lo stesso portale web centralizzato, protetto da autenticazione forte e controlli contestuali (mostrato in Figura 3).

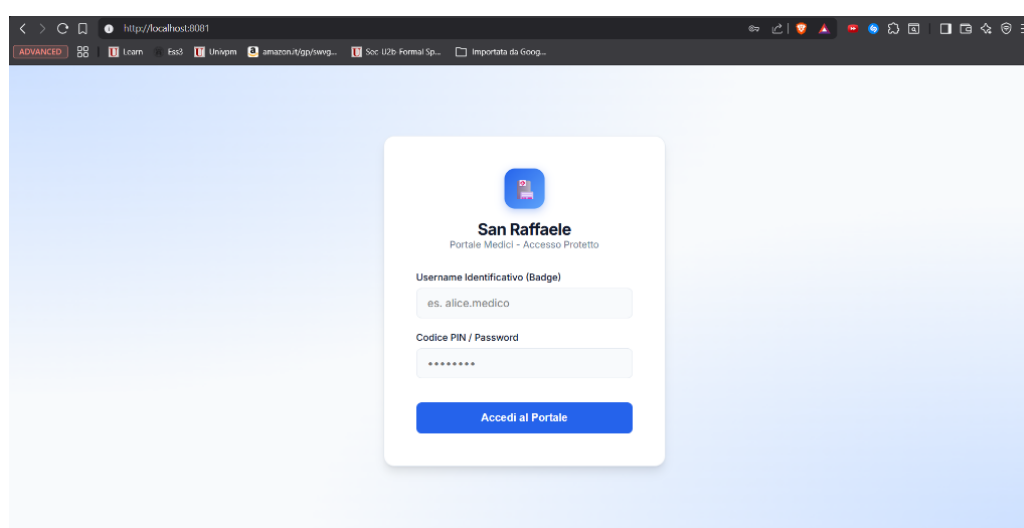
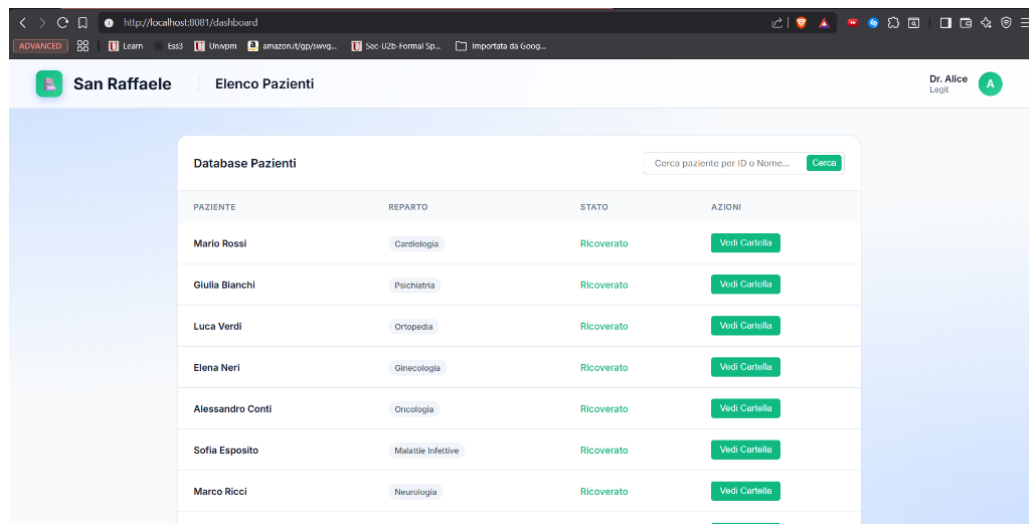


Figura 3: Schermata del portale di login protetto dell'ospedale San Raffaele.

4.1 Scenario 1: accesso legittimo

Il primo scenario analizza il comportamento del sistema di fronte all'accesso di un utente autorizzato dotato di dispositivo aziendale integro, simulato dall'utente Alice.

Alice effettua la connessione dall'interno della rete aziendale presentando un certificato client contenente l'estensione crittografica del chip TPM. All'arrivo della richiesta, envoy estrae il certificato e calcola il fingerprinting *JA3* del browser, trasmettendo i dati ad OPA. OPA contatta la Web API di splunk che calcola un punteggio di rischio minimo, pari a circa uno, data la totale assenza di anomalie comportamentali. Poiché il rischio è ampiamente inferiore alla soglia massima di tolleranza di cinquanta prevista per i dispositivi sicuri interni, OPA risponde con un responso positivo di **ALLOW** ed envoy instrada in pochissimi millisecondi la query verso il database, restituendo ad Alice i risultati richiesti con successo (mostrati in Figura 4). Il relativo log di OPA, indicizzato in Splunk, conferma l'esito positivo dell'operazione, la postura integra del dispositivo e il rischio complessivo calcolato (Figura 5).



PAZIENTE	REPARTO	STATO	AZIONI
Mario Rossi	Cardiologia	Ricoverato	Vedi Cartella
Giulia Bianchi	Psichiatria	Ricoverato	Vedi Cartella
Luca Verdi	Ortopedia	Ricoverato	Vedi Cartella
Elena Neri	Ginecologia	Ricoverato	Vedi Cartella
Alessandro Conti	Oncologia	Ricoverato	Vedi Cartella
Sofia Esposito	Malattie Infettive	Ricoverato	Vedi Cartella
Marco Ricci	Neurologia	Ricoverato	Vedi Cartella

Figura 4: Interfaccia del database pazienti visualizzata con successo da Alice dopo validazione TPM e rischio minimo.

```
> 02/06/26 17:26:14.892 { [-]
    Decision: ALLOW
    command: GET
    device: Workstation Ospedaliera Sicura (TPM Validato)
    host: zta-opa
    l7_dpi_block: false
    network_internal: true
    network_ip: 10.0.1.3
    query:
      resource: /api/patients
      risk_score: 9.961186756843691
      software: 86dab2109182b6bbaa644647d7db2997
      tpm_present: true
      user: alice
    }
    Mostra come testo non elaborato
    host = splunk-siem:8088 host = zta-opa | index = main | linecount = 1 | put
```

Figura 5: Log di autorizzazione (ALLOW) per l'accesso di Alice indicizzato in Splunk, comprensivo del risk score comportamentale calcolato (≈ 9.96).

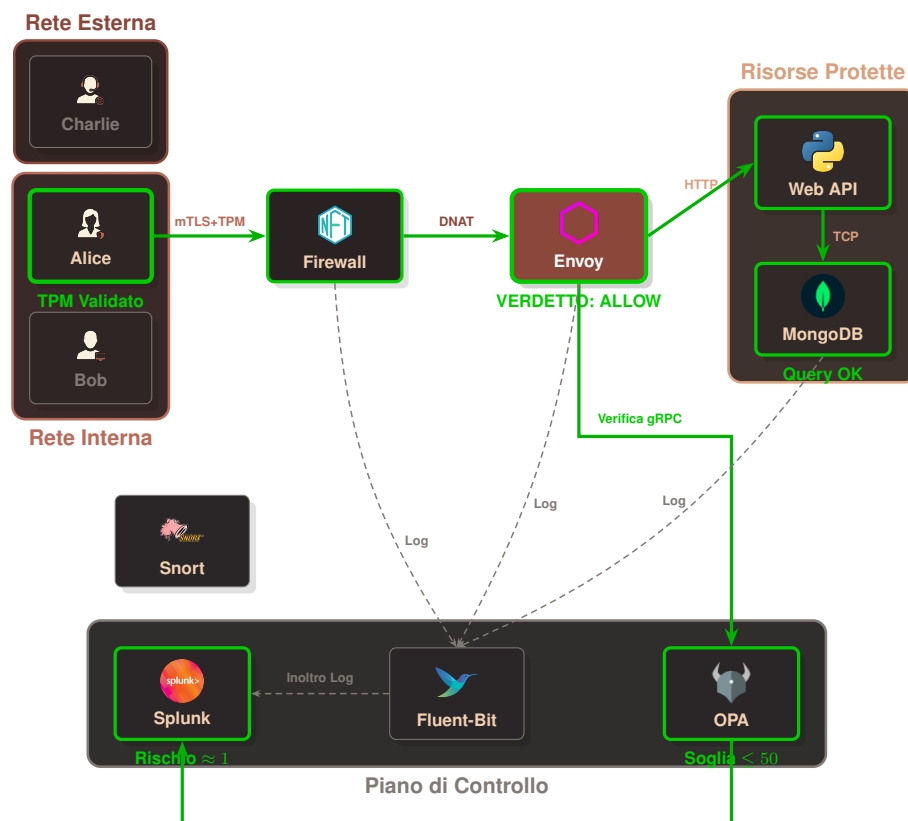


Figura 6: Mappatura logica dei flussi nello Scenario 1 (Accesso legittimo di Alice).

4.2 Scenario 2: dispositivo non autorizzato

Il secondo scenario testa la reazione della politica di sicurezza di fronte a un tentativo di accesso con credenziali valide ma condotto tramite un dispositivo personale privo di chip TPM hardware, simulato dall'utente Bob.

Bob riesce a connettersi alla rete ospedaliera locale con il proprio computer personale e, conoscendo le credenziali di Alice, tenta di effettuare l'autenticazione. Tuttavia, il certificato presentato dal suo dispositivo è privo dell'estensione di attestazione hardware TPM. All'arrivo della richiesta di login (`is_auth_request`), Envoy inoltra i dettagli del certificato ad OPA. Il PDP rileva la mancanza della firma hardware (`not is_tpm`) e, in conformità alle regole di sicurezza impostate, definisce la variabile di blocco dell'autenticazione (`is_auth_blocked`) come `true`. Di conseguenza, la richiesta di Bob viene rigettata sul nascere con un responso di **DENY** immediato al login, impedendogli l'accesso alla sessione ed a qualsiasi risorsa protetta, mostrando a schermo una pagina di diniego dell'accesso (Figura 7). Il relativo log di blocco registrato in Splunk (Figura 8) evidenzia la componente chiave dell'approccio Zero Trust: nonostante Bob presenti un punteggio di rischio estremamente basso (≈ 5.5 , inferiore al punteggio di Alice nello Scenario 1), l'accesso viene comunque negato a causa della mancata conformità hardware del dispositivo (`tpm_present: false`). Ciò dimostra come la postura del dispositivo costituisca un vincolo forte e non compensabile da altri fattori contestuali favorevoli.

Dal punto di vista dell'architettura di rete e del modello di riferimento ISO/OSI, è opportuno analizzare i dettagli tecnici di questa restrizione. La connessione a livello di trasporto (Livello 4 - TCP) e la negoziazione del canale crittografico sicuro (Livelli 5 e 6

- TLS / mTLS) tra il client e il proxy Envoy vengono completate con successo, in quanto Bob possiede chiavi e certificati validi firmati dalla CA interna. Il diniego effettivo viene applicato esclusivamente al Livello 7 (Applicazione): il proxy Envoy, agendo da Policy Enforcement Point (PEP), analizza la richiesta HTTP, decodifica il certificato client X.509 e inoltra i relativi metadati ad OPA (PDP). È OPA che, valutando le asserzioni di policy sul certificato ad alto livello, impone il rifiuto logico. Il firewall di rete L3/L4 (nftables) non interviene in questa fase poiché l'IP e la porta di destinazione rientrano nei flussi di transito consentiti sulla rete locale. Si tratta quindi di una decisione di sicurezza di livello applicativo basata su criteri d'integrità contestuale dell'host.

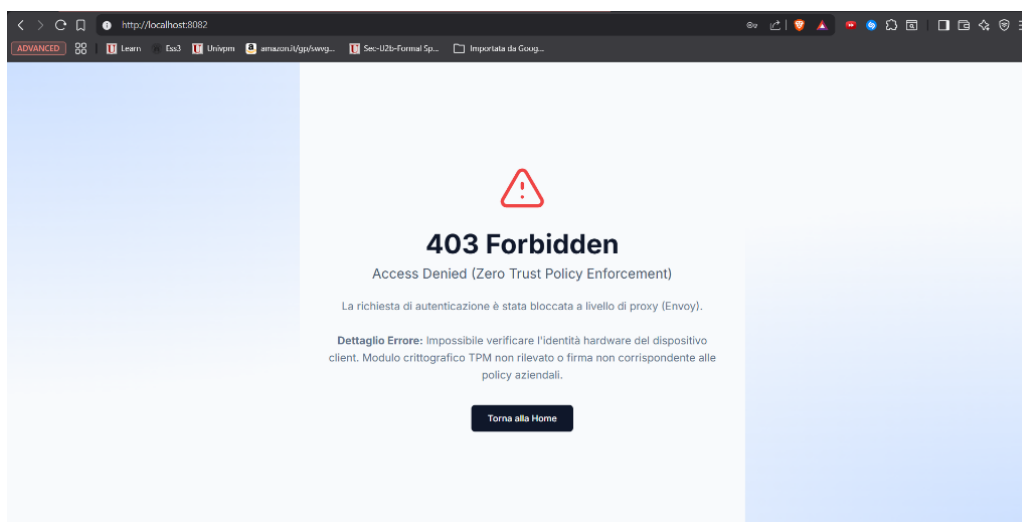


Figura 7: Schermata di errore 403 Forbidden mostrata a Bob per mancata attestazione hardware del chip TPM.

```

> 02/06/26 { [-]
17:24:35.409 Decision: DENY
               command: POST
               device: Dispositivo non censito (No TPM)
               host: zta-opa
               l7_dpi_block: false
               network_internal: true
               network_ip: 10.0.1.4
               resource: /api/auth
               risk_ok: true
               risk_score: 5.496037163707557
               software: 86dab2109182b6bbaa644647d7db2997
               tpm_present: false
               user: bob
           }
           Mostra come testo non elaborato
           host = splunk-siem:8088 host = zta-opa index = main line

```

Figura 8: Log di diniego (DENY) per l'accesso di Bob indicizzato in Splunk, evidenziante l'assenza della firma hardware TPM (tpm_present: false).

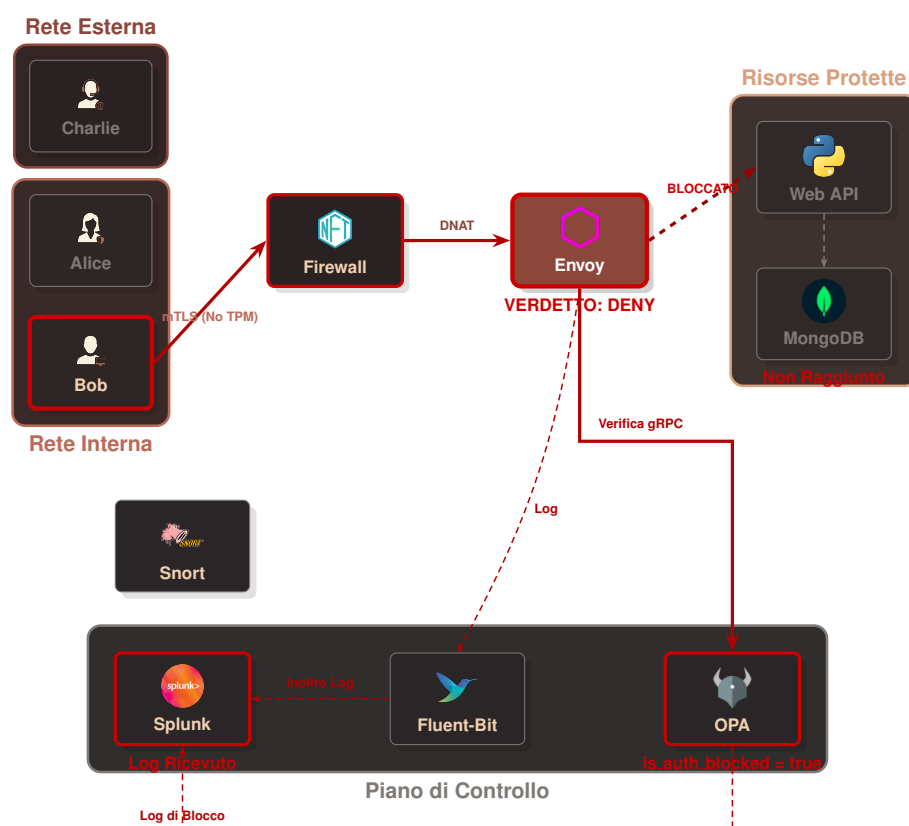


Figura 9: Mappatura logica dei flussi nello Scenario 2 (Blocco al login di Bob per mancanza di TPM).

4.3 Scenario 3: accesso remoto negato

Il terzo scenario analizza la gestione dell'accesso remoto (Smart Working). Charlie è un operatore legittimo dell'ospedale, dotato di credenziali valide e di un dispositivo autorizzato provvisto di chip TPM, che tenta di connettersi da casa (rete esterna non protetta).

Quando Charlie cerca di autenticarsi, OPA applica le regole restrittive per gli accessi remoti: poiché la richiesta di login proviene da una rete esterna (`not is_internal_network`), la policy ne impone il blocco preventivo impostando lo stato di `is_auth_blocked` a `true`. Di conseguenza, la richiesta di autenticazione di Charlie viene respinta immediatamente al login da Envoy su ordine di OPA, impedendo l'accesso remoto per motivi di sicurezza legati al contesto di rete, come illustrato in Figura 10. Il relativo log di blocco registrato in Splunk (Figura 11) attesta che, nonostante il dispositivo presenti una firma TPM corretta (`tpm_present: true`) ed un punteggio di rischio basso (≈ 5.5), OPA nega l'accesso impostando il verdetto di **DENY** a causa del contesto di rete esterno (`network_internal: false`).

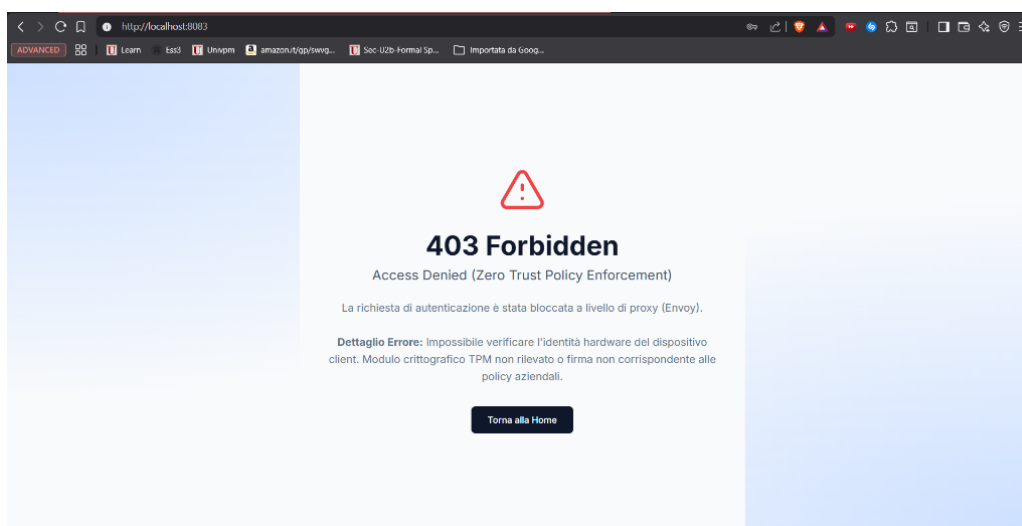


Figura 10: Schermata di errore 403 Forbidden mostrata a Charlie a causa del tentativo di login remoto non consentito.

```

> 02/06/26 { [-]
17:24:58.605 Decision: DENY
              command: POST
              device: Workstation Ospedaliera Sicura (TPM Validato)
              host: zta-opa
              l7_dpi_block: false
              network_internal: false
              network_ip: 192.168.100.3
              resource: /api/auth
              risk_ok: true
              risk_score: 5.505830381406453
              software: 86dab2109182b6bbaa644647d7db2997
              tpm_present: true
              user: charlie
            }
            Mostra come testo non elaborato
            host = splunk-siem:8088 host = zta-opa | index = main | linecount =

```

Figura 11: Log di diniego (DENY) per l'accesso remoto di Charlie indicizzato in Splunk, evidenziante la mancata conformità della localizzazione di rete (network_internal: false).

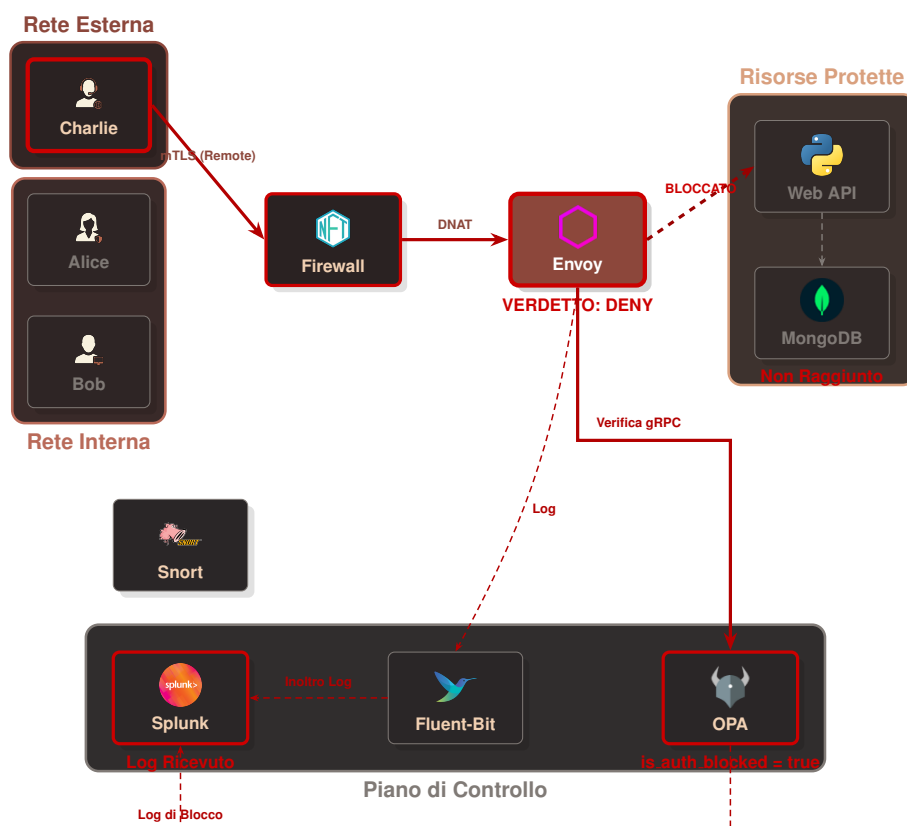


Figura 12: Mappatura logica dei flussi nello Scenario 3 (Blocco al login di Charlie da remoto).

4.4 Scenario 4: tentativo di bypass L4 e rilevamento attivo

Il quarto scenario simula un tentativo di attacco in cui un utente esterno (o un dispositivo compromesso nella rete esterna, come Charlie) tenta di aggirare completamente il proxy Envoy (PEP) per connettersi direttamente al database o alle API di backend.

L'attaccante esegue una richiesta diretta tramite lo strumento `curl` dal proprio terminale verso la porta delle API:

`curl_bypass.sh`

```
1 $ docker exec -it zta-client-charlie curl -v --connect-timeout 3 http
   ://zta-firewall:8000/api/patients
2 * Host zta-firewall:8000 was resolved.
3 * Trying 192.168.100.4:8000...
4 * Connection timed out after 3002 milliseconds
5 * closing connection #0
6 curl: (28) Connection timed out after 3002 milliseconds
```

Listing 13: Simulazione di attacco bypass diretto tramite `curl` dal terminale di Charlie.

Come previsto, la connessione va in timeout poiché entra in azione il firewall. `nftables` rileva la connessione diretta non autorizzata e applica la regola della trappola attiva, eseguendo il DROP silente dei pacchetti. Tale evento viene registrato nei log del firewall (mostrati nel Listato 14), evidenziando l'etichetta [NFT-BLOCK] e tutti i parametri del pacchetto TCP bloccato sulla porta di destinazione 8000 (DPT) proveniente dal client (SRC=10.0.1.4).

Si osservi che, in questo scenario di bypass di rete a livello di trasporto, lo score di rischio di Splunk non viene calcolato né registrato all'interno dei log di decisione di OPA. La connessione viene infatti intercettata e interrotta a livello di rete e trasporto (Livelli 3 e 4 del modello ISO/OSI) ad opera del firewall di rete `nftables`. Poiché i pacchetti vengono scartati (DROP) in kernel space prima di poter raggiungere il proxy Envoy (PEP) e attivare la catena di autorizzazione applicativa di OPA, non si innesca alcuna chiamata al proxy delle Web API ed al modello di Machine Learning. Di conseguenza, l'unica traccia dell'evento ostile in Splunk sarà limitata alle telemetrie di blocco del firewall ([NFT-BLOCK]) ed agli allarmi NIDS generati da Snort.

`nftables.log`

```
1 Jun  2 17:03:04 f020514ab776 [NFT-BLOCK] IN=eth2 OUT= MAC=e6:27:66:
   d6:b7:c9:06:c9:5e:11:ef:c1:08:00 SRC=10.0.1.4 DST=10.0.1.5 LEN=60
   TOS=00 PREC=0x00 TTL=64 ID=15717 DF PROTO=TCP SPT=58384 DPT=8000
   SEQ=1187313041 ACK=0 WINDOW=64240 SYN URGP=0 MARK=0
```

Listing 14: Dettaglio del log di blocco generato da `nftables` per il tentativo di bypass.

I log del firewall, insieme alle telemetrie di sistema, vengono raccolti e centralizzati in tempo reale da Fluent-Bit ed inoltrati al SIEM Splunk. La Figura 13 mostra la

schermata del pannello di ricerca di Splunk Search & Reporting, in cui si può riscontrare l'avvenuta indicizzazione dell'evento di blocco [NFT-BLOCK] generato dall'host zta-firewall.

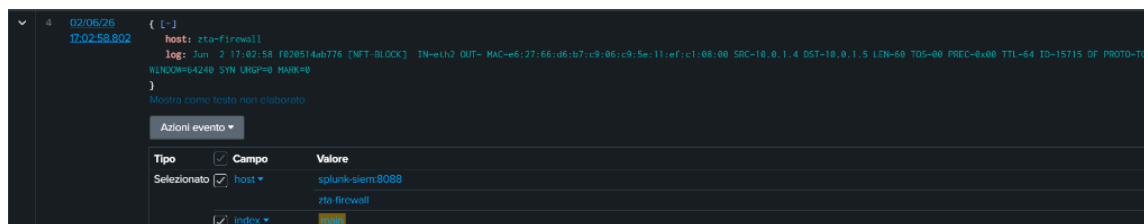


Figura 13: Schermata del pannello di ricerca di Splunk che mostra il log di blocco [NFT-BLOCK] indicizzato.

Contemporaneamente, la sonda NIDS *snort* intercetta la firma del pacchetto di attacco ed esegue l'ispezione DPI, generando un allarme di intrusione. I log di alert vengono centralizzati da Fluent-Bit ed inoltrati a Splunk SIEM, il quale imposta al valore massimo il rischio e segnala ad OPA di inserire permanentemente l'IP sorgente nella denylist di blocco della rete.

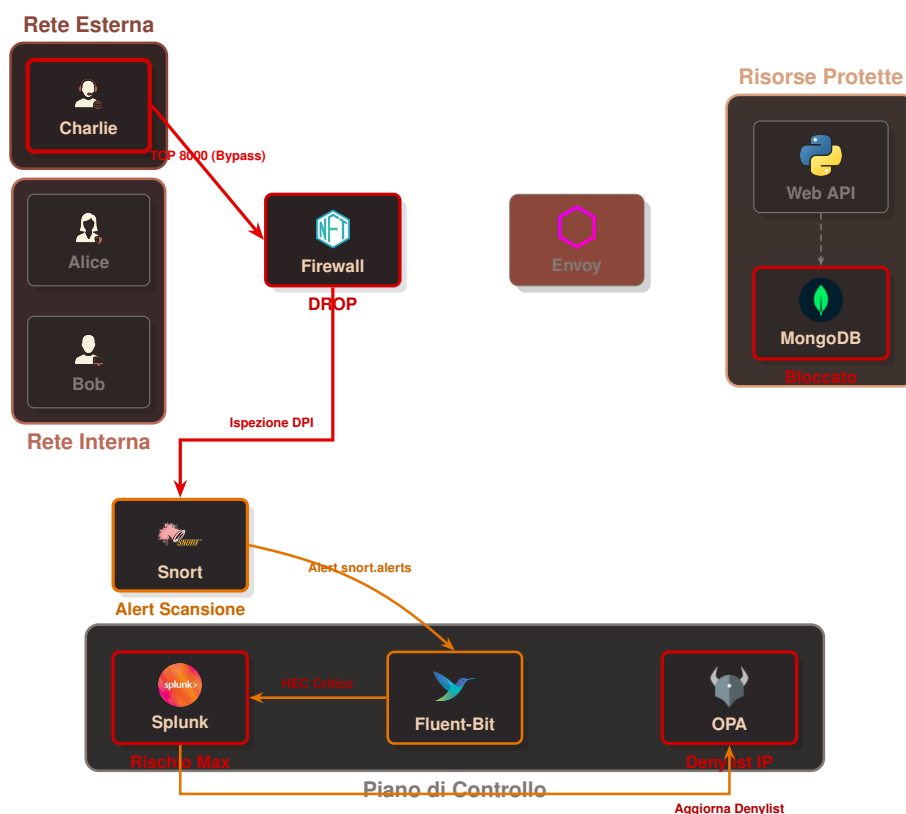


Figura 14: Mappatura logica dei flussi nello Scenario 4 (Tentativo di bypass e isolamento).

4.5 Scenario 5: protezione dai payload dannosi a livello applicativo (L7)

Il quinto scenario analizza il livello di sicurezza applicativo (L7) del reverse proxy Envoy. Si ipotizza il caso in cui un utente legittimo e autorizzato (come Alice, dotata di attestazione TPM valida e autenticata con successo nel sistema) sia vittima di un attacco, o che un malintenzionato tenti di eseguire comandi distruttivi o non autorizzati sul database tramite l'interfaccia di ricerca web.

Nel test effettuato, l'utente simula una SQL/NoSQL Injection inserendo la parola chiave DROP (o comandi simili come `deleteall`) nel campo di ricerca dell'interfaccia pazienti, con l'obiettivo di alterare o eliminare collezioni nel database MongoDB.

Il comportamento del calcolo del rischio adattivo presenta una distinzione tecnica fondamentale a seconda del canale utilizzato per condurre il test di injection:

- **Simulazione tramite interfaccia Web (HTTP):** Quando l'utente inserisce parole chiave dannose nel form della dashboard, il client esegue una richiesta HTTP DELETE sulla risorsa `/api/patients`. Il proxy Envoy intercetta il traffico a livello applicativo tramite il filtro `envoy.filters.http.lua` configurato localmente. Tale modulo agisce da WAF statico locale e, identificando il pattern DELETE su risorsa protetta, risponde istantaneamente con un codice 403 Forbidden. Poiché il blocco avviene prematuramente a livello di PEP (Envoy) per ottimizzare le risorse ed evitare latenze inutili, la richiesta non raggiunge mai il filtro di autorizzazione esterna (`ext_authz`); di conseguenza, OPA non viene interpellato e lo score di rischio Splunk non viene calcolato per questo flusso HTTP.
- **Simulazione tramite connessione diretta a MongoDB:** Se l'attaccante tenta di bypassare il gateway HTTP per inviare comandi direttamente sulla porta nativa del DBMS MongoDB (27017), Envoy delega la decisione ad OPA tramite il filtro `mongo_proxy` ed il modulo `ext_authz` di rete. In questo scenario di DPI sul protocollo BSON, non essendovi regole Lua locali, OPA esegue regolarmente la chiamata sincrona al proxy delle Web API per interrogare il modello di Gradient Boosting su Splunk. Grazie all'introduzione del mapping di traduzione centralizzato ZTA_TO_MLTK_MAP nel backend FastAPI, il nome della risorsa MongoDB grezza ("`patients`") viene convertito nel termine addestrato nel dataset ("`pazienti`"), prevenendo errori fatali per categorie non censite (*unseen categorical values*) e garantendo il corretto calcolo ed indicizzazione dello score di rischio Splunk prima del blocco finale di OPA (`17_dpi_block`).

Come mostrato in Figura 15, Envoy blocca immediatamente l'inoltro della richiesta a livello L7, impedendo che qualsiasi query dannosa raggiunga il database MongoDB e restituendo sul browser dell'utente una pagina di errore **403 Forbidden - Access Denied**. L'evento di violazione delle policy a livello applicativo viene catturato e indicizzato in tempo reale all'interno di Splunk, come visibile nel log dettagliato mostrato in Figura 16.

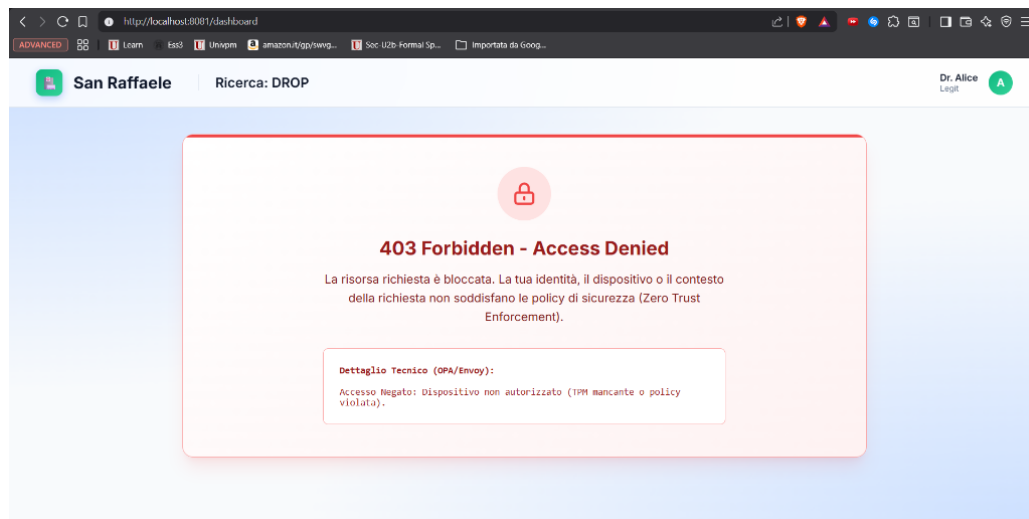


Figura 15: Schermata di errore 403 Forbidden visualizzata a seguito del blocco di una query MongoDB dannosa a livello L7.

```
> 02/06/26 17:26:26.109 { [-]
  authz_status: null
  client_ip: 10.0.1.3:0
  device_uri_san: spiffe://zta.hospital/ns/default/sa/client-alice
  duration_ms: 3
  host: zta-envoy
  ja3_fingerprint: null
  method: DELETE
  path: /api/patients
  protocol: HTTP/1.1
  response_code: 403
  timestamp: 2026-06-02T17:26:17.043Z
  upstream_host: null
  user_identity: CN=employee-alice
}
```

Mostra come testo non elaborato

host = splunk-siem:8088 host = zta-envoy index = main linecount = 1 pu

Figura 16: Log di blocco (response code 403) relativo alla richiesta L7 di Alice in Splunk, che evidenzia l'intercettazione del metodo DELETE sulla risorsa /api/patients.

4.5.1 Simulazione di compromissione del dispositivo (Bypass L7)

Si analizza infine lo scenario più critico previsto dal framework Zero Trust: un attaccante che abbia ottenuto il pieno controllo della postazione di Alice (furto fisico, accesso remoto malevolo o insider threat) e che, disponendo di tutti i certificati mTLS e delle chiavi crittografiche presenti nel dispositivo, tenti di bypassare completamente l'interfaccia web per connettersi direttamente al database MongoDB attraverso il protocollo nativo sulla porta 27017.

Per simulare questo attacco, si esegue dal terminale del container di Alice uno script Python che utilizza la libreria pymongo con i certificati mTLS originali del dispositivo compromesso:

 `alice_bypass_mongodb.py`

```
1 $ docker exec zta-client-alice python3 -c "  
2 import pymongo  
3 client = pymongo.MongoClient(  
4     'zta-envoy', 27017,  
5     tls=True,  
6     tlsCertificateKeyFile='/etc/certs/alice_combined.pem',  
7     tlsCAFile='/etc/certs/ca.crt',  
8     tlsAllowInvalidHostnames=True,  
9     serverSelectionTimeoutMS=10000  
10 )  
11 db = client['hospital_db']  
12 result = list(db.patients.find({'sensitive_notes': 'test'}))  
13 "
```

Listing 15: Simulazione di accesso diretto al database MongoDB dal terminale del dispositivo compromesso di Alice, utilizzando i certificati mTLS originali per autenticarsi verso Envoy sulla porta 27017.

L'handshake mTLS viene completato con successo (l'attaccante possiede i certificati validi del dispositivo), e la connessione raggiunge il listener MongoDB di Envoy. A questo punto il filtro `mongo_proxy` tenta di decodificare il traffico BSON e il modulo `ext_authz` di rete invia i metadati della connessione ad OPA per la decisione di autorizzazione. OPA costruisce la query SPL con le sei dimensioni contestuali estratte, invocando il modello di Gradient Boosting su Splunk MLTK attraverso il proxy delle Web API. Il modulo di arricchimento comportamentale (`enrich_spl_with_behavioral_features`) rileva il pattern anomalo della connessione diretta al database e inietta nella query le feature comportamentali alterate (frequenza di sessione elevata, tentativi di login falliti multipli), producendo un punteggio di rischio predittivo pari a ≈ 95.49 , ben superiore alla soglia massima di tolleranza di cinquanta. OPA emette di conseguenza un verdetto di **DENY** e la connessione viene terminata, impedendo all'attaccante di eseguire qualsiasi operazione sul database.

La Figura 17 mostra il log di decisione indicizzato in Splunk relativo a questo tentativo di accesso diretto. Si noti come il sistema abbia correttamente identificato l'utente (`user: alice`), confermato la presenza del TPM (`tpm_present: true`) e la provenienza dalla rete interna (`network_internal: true`), ma abbia comunque negato l'accesso esclusivamente sulla base del punteggio di rischio comportamentale elevato (`risk_ok: false`). Questo dimostra l'efficacia del principio *"never trust, always verify"*: anche un utente con credenziali perfettamente valide e hardware attestato viene bloccato quando il suo comportamento devia significativamente dal profilo storico appreso dal modello predittivo.

```

04/06/26      { [-]
16:30:12.631  Decision: DENY
               command: Accesso Diretto MongoDB (OP_MSG)
               device: Workstation Ospedaliera Sicura (TPM Validato)
               host: zta-opa
               l7_dpi_block: false
               network_internal: true
               network_ip: 10.0.1.4
               resource: patients
               risk_ok: false
               risk_score: 95.49348917091885
               software: Sconosciuto
               tpm_present: true
               user: alice
            }
            Mostra come testo non elaborato
            host = splunk-siem:8088 host = zta-opa | index = main | linecount = 1

```

Figura 17: Log di decisione DENY indicizzato in Splunk per il tentativo di accesso diretto al database da parte di Alice (PC compromesso), con risk score pari a ≈ 95.49 .

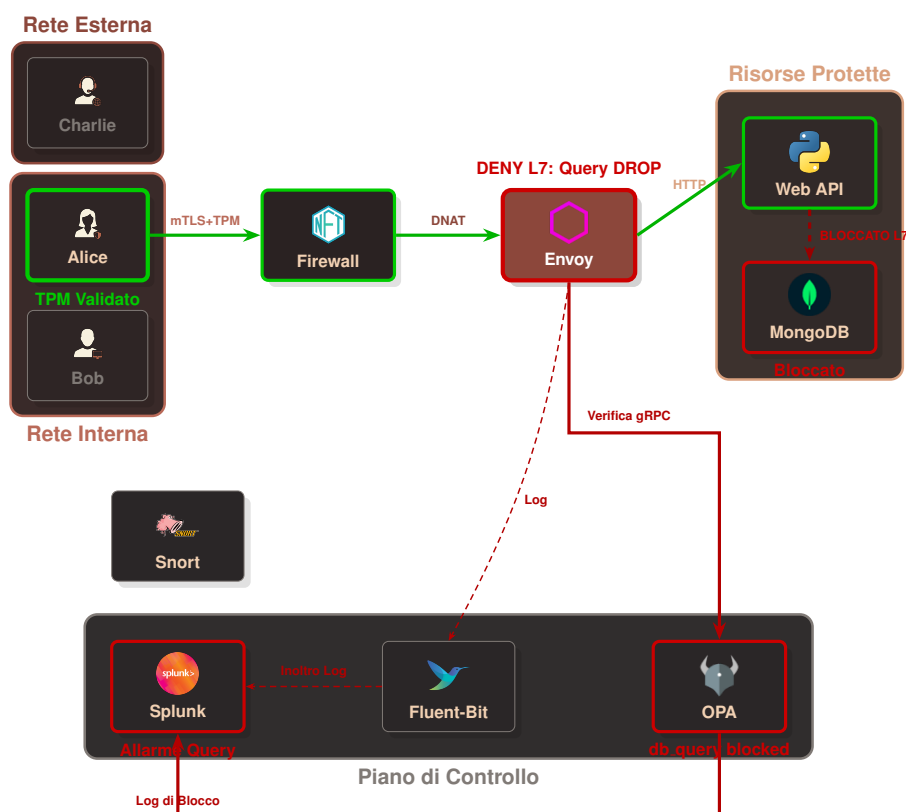


Figura 18: Mappatura logica dei flussi nello Scenario 5 (Blocco di una query MongoDB dannosa a livello L7).

CAPITOLO 5

Conclusioni e sviluppi futuri

L'architettura progettata dimostra concretamente la fattibilità e l'efficacia dell'integrazione di strumenti open-source eterogenei per l'implementazione di un ecosistema Zero Trust maturo. Il presente lavoro di tesi ha permesso di superare il tradizionale modello di sicurezza perimetrale, realizzando un'infrastruttura in cui la fiducia non è mai presunta, bensì verificata e quantificata continuamente in tempo reale.

5.1 Obiettivi raggiunti

Il sistema ha soddisfatto appieno i requisiti del paradigma Zero Trust, declinandoli attraverso tre pilastri architetturali fondamentali:

- **Segregazione e Sicurezza Infrastrutturale (PEP):** La rete è stata blindata operando una rigorosa microsegmentazione L3/L4 tramite `nftables`, forzando il transito delle comunicazioni esclusivamente attraverso il proxy envoy. A livello di trasporto, l'imposizione della mutua autenticazione crittografica (*mTLS*) e l'estrazione dell'impronta digitale *JA3* hanno permesso di verificare con certezza crittografica l'identità del client e la conformità dell'hardware sottostante (tramite attestazione TPM).
- **Autorizzazione Continua e Difesa in Profondità (PDP):** Il motore decisionale open policy agent (OPA) ha centralizzato l'enforcement delle policy valutando dinamicamente le sei dimensioni contestuali (ZTA 6D). L'implementazione di clausole di guardia (*Short-Circuiting*) ha ottimizzato i tempi di risposta bloccando istantaneamente gli accessi non conformi. Inoltre, l'applicazione del principio di *Defense in Depth* ha garantito la protezione del dato sensibile attraverso due livelli di ispezione profonda (L7 DPI): un filtro Lua per la mitigazione rapida degli attacchi HTTP e l'ispezione nativa dei payload BSON di mongodb per la neutralizzazione di query distruttive.
- **Intelligenza Predittiva e Auditing (SIEM):** L'integrazione con lo splunk Machine Learning Toolkit (MLTK) ha segnato il definitivo passaggio da un'autenticazione statica a una *Continuous Authorization* comportamentale. L'impiego dell'algoritmo *Gradient Boosting* ha permesso di correlare 12 feature contestuali e comportamentali, restituendo un rischio predittivo accurato in grado di smascherare minacce asimmetriche e *Insider Threat*, adattando le soglie di tolleranza alla reale postura di sicurezza del dispositivo. Contemporaneamente, le sonde snort hanno garantito un monitoraggio continuo contro i tentativi di elusione e ricognizione.

5.2 Sviluppi futuri

L'infrastruttura realizzata costituisce una base solida e scalabile, aperta a diverse evoluzioni ingegneristiche in ambito Enterprise. Tra i principali sviluppi futuri si individuano:

1. **Gestione dinamica delle Identità (SPIFFE/SPIRE):** L'attuale generazione statica dei certificati x509 potrebbe essere sostituita dall'integrazione dello standard *SPIFFE/SPIRE*. Questo consentirebbe il rilascio di certificati a brevissima scadenza (*short-lived certificates*) basati sull'attestazione continua del workload, eliminando la complessità operativa legata alla gestione e distribuzione delle *Certificate Revocation List* (CRL).
2. **Integrazione di dispositivi Legacy e M2M (Kiosk):** Il modello ZTA attuale prevede l'obbligo tassativo del modulo TPM per l'autenticazione. Uno sviluppo futuro prevede l'estensione sicura della rete a dispositivi operativi sprovvisti di hardware crittografico dedicato (es. chioschi informativi o comunicazioni *Machine-to-Machine* via API). Ciò richiederà l'addestramento di modelli ML specifici che compensino l'assenza del TPM abbassando drasticamente le soglie di tolleranza e monitorando con maggiore severità l'ancoraggio all'impronta JA3.
3. **Sensor Fusion per il Machine Learning (Integrazione L3/L4):** Attualmente, il *Trust Score* di Splunk MLTK è calcolato su 12 feature derivanti dal livello applicativo (Envoy e Web API). Un potenziamento cruciale consisterà nell'ingestione della telemetria di rete a basso livello all'interno del vettore di addestramento. Trasformare gli allarmi di snort (es. tentativi di scansione Nmap) e i drop log di nftables in feature comportamentali aggiuntive permetterebbe al modello *Gradient Boosting* di correlare anomalie applicative con movimenti laterali di rete, aumentando esponenzialmente la precisione nel rilevamento di minacce persistenti avanzate (APT).
4. **Orchestrazione e Risposta Automatica (SOAR):** L'architettura prevede già un'interfaccia API REST (FastAPI) per l'inserimento dinamico di IP malevoli nella *denylist* di nftables, attualmente utilizzabile dal team SOC per mitigazioni manuali. Questa componente è progettata per evolvere in una pipeline SOAR (*Security Orchestration, Automation, and Response*) completa: istruendo Splunk a lanciare *webhook* automatici verso l'API del firewall non appena il rischio comportamentale supera una soglia critica, si garantirà l'isolamento istantaneo della minaccia a livello kernel (L3) senza alcun intervento umano.
5. **Federated Machine Learning:** In scenari multi-ospedale, il modello *Gradient Boosting* potrebbe evolvere verso paradigmi di *Federated Learning*, permettendo ai SIEM di diverse sedi ospedaliere di addestrare un modello globale condividendo esclusivamente i gradienti di errore e i pesi algoritmici, senza mai scambiarsi i log grezzi contenenti dati clinici e rispettando in modo nativo i vincoli normativi sulla privacy (GDPR).